



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

Rediseño e implementación de manipuladores para
un robot de servicio de propósito general

TESIS

Que para obtener el título de
Ingeniero Mecatrónico

P R E S E N T A N

Rafael Cuéllar Ramírez
Marco Elian Soriano Pimentel

DIRECTOR DE TESIS

Dr. Jesús Savage Carmona



Ciudad Universitaria, Cd. Mx., 2026

Índice general

1. Introducción	4
1.1. Robótica de servicio doméstico	5
1.2. Justificación	5
1.3. Planteamiento del problema	6
1.4. Alcance	6
1.5. Objetivos	7
1.5.1. Objetivos específicos	7
1.6. Organización del trabajo	7
2. Antecedentes	9
2.1. <i>Justina</i>	9
2.2. Diseño de manipuladores robóticos	13
2.2.1. Definición	13
2.2.2. Movilidad	13
2.2.3. Criterio de Grashof	14
2.2.4. Síntesis cinemática	15
2.2.5. Análisis cinemático	16
2.3. Variables de interés e instrumentación	17
2.4. Control de manipuladores robóticos	17
3. Metodología	18
3.1. Identificación de necesidades	19
3.1.1. Perfiles de usuario	19
3.1.2. Recopilación de datos	20
3.1.3. Interpretación y organización jerárquica	20
3.1.4. Importancia relativa y selección	20
3.2. Establecimiento de especificaciones objetivo	21
3.2.1. Traducción de necesidades a métricas	21
3.2.2. Benchmarking y análisis competitivo	22
3.2.3. Definición de valores objetivo	23
3.3. Generación de conceptos	23
3.3.1. Descomposición funcional del problema	24
3.3.2. Búsqueda externa de soluciones	24

3.3.3. Exploración sistemática y acotamiento del espacio de soluciones	24
3.3.4. Desarrollo de conceptos completos	24
A. Justina	26
A.0.1. Executor	26
A.0.2. HRI	29
A.0.3. Hardware	35
A.0.4. Knowledge	43
A.0.5. Manipulation	44
A.0.6. Navigation	47
A.0.7. Planning	53
A.0.8. Supervisor	57
A.0.9. Vision	60
B. Protocolo de investigación	67
C. Carta de consentimiento informado	73

Capítulo 1

Introducción

En 1920, el término **robot**, derivado de la palabra eslava *robota*, que significa trabajo subordinado, fue introducido por el dramaturgo checo Karel Čapek en su obra *Rossum's Universal Robot* [1]. Sin embargo, es hasta 1961 cuando se inicia la robótica como disciplina con la invención de los brazos autónomos para asistir a la manufactura en líneas de producción con el robot *Unimate* [2]. Sus tareas se centraban en la manipulación de piezas y ensambles desde una posición fija en un entorno controlado, prácticamente estático, con la intención de relevar a los trabajadores de tareas monótonas e incrementar la producción.

Debido a esta reciente introducción e investigación en robótica, aún existen diversas definiciones de **robot**, que varían de acuerdo con el campo de aplicación y el área de investigación. Derivado del enfoque industrial inicial, las definiciones tanto de la *International Standard Organization* en el estándar ISO 8373¹ como de la *Robot Institute of America* reducen el término de robot a un manipulador [3]. Por otro lado, Bajd [4] amplía la definición, considerando la robótica contemporánea, como la rama de la ciencia que estudia a aquellos sistemas cuya principal característica es el movimiento.

Durante las últimas décadas se han diseñado y construido una gran cantidad de robots en dos ejes principales: su grado de autonomía, entendida como la habilidad de tomar decisiones basadas en una representación interna del mundo, y su capacidad de actuar en entornos complejos. Ambos relacionados directamente con su inteligencia [2].

De esta forma, la definición que resulta más útil para el desarrollo de este trabajo es la de Arkin [5], quien propone que un robot inteligente es una máquina capaz de extraer información de su ambiente y usar el conocimiento acerca de su mundo para moverse de manera segura y significativa, con un propósito específico.

¹El estándar ISO 8373 define robot como: “Un manipulador automáticamente controlado, reprogramable, multiuso, programable en tres o más ejes, que pueden estar fijos en un lugar o movilizarse para ser usado en aplicaciones de automatización industrial”; mientras que la RIA considera a un robot como “un manipulador multifuncional reprogramable diseñado para mover materiales, partes, herramientas o dispositivos especializados a través de diversos movimientos programados para la realización de diversas tareas”.

1.1. Robótica de servicio doméstico

Los avances exponenciales en robótica han abierto la posibilidad de introducir robots en nuevos entornos. El objetivo de la robótica de servicio es reducir o eliminar el trabajo físico de las personas en lugares como casas. Los dispositivos robóticos tienen el potencial de ayudar a las personas con discapacidad a moverse o brindar compañía a ancianos. Adicionalmente, los robots pueden hacer más seguros los espacios con acciones que van desde recoger objetos para evitar tropiezos hasta verificar que se haya cerrado totalmente la llave de gas [3].

La definición de esta rama más reciente complementa la de Arkin como una máquina móvil reprogramable autónoma o semiautónoma diseñada para operar en entornos dinámicos de manera segura, confiable y robusta, y ser capaz de realizar tareas específicas [3].

Bill Gates [6] vaticinó que en un hogar promedio habría robots de servicio doméstico con propósito específico encargados de cortar el césped, vigilar, limpiar, aspirar, trapear, lavar y planchar, como ya los hay en la actualidad. Pero también habría **robots humanoides de propósito general** que ayudarían a las personas a traer y llevar objetos. Este último tipo de robot tendría la capacidad de comunicarse con los miembros de la casa utilizando lenguaje natural y planear las acciones y movimientos necesarios con base en sus núcleos de acción predefinidos [3] para realizar sus funciones.

De esta forma, los robots de servicio doméstico deben contar con tres habilidades básicas: una navegación robusta, manipulación móvil y comunicación intuitiva con el usuario [7].

En 1997, a partir de avances significativos en inteligencia artificial y robótica, tales como la derrota del campeón mundial de ajedrez Gary Kasparov a manos del *IBM Deep Blue* y el despliegue del primer sistema robótico autónomo en Marte *Sojourner*, surgió la RoboCup (Robot World Cup), una de las iniciativas con mayor relevancia a nivel internacional en el desarrollo de estos campos. Con el objetivo de definir metas a largo plazo y estandarizar plataformas de desarrollo, se coordinan anualmente las principales instituciones enfocadas en investigación alrededor del mundo para presentar sus avances [8].

Debido al éxito de este proyecto, el alcance de la RoboCup se amplió con el surgimiento de diferentes ligas. RoboCup@HOME se fundó con el objetivo de evaluar las capacidades de la tecnología emergente aplicada a la mejora de las habilidades básicas en robótica de servicio [9].

1.2. Justificación

La manipulación es un problema bien conocido en robótica. Constituye la interfaz de interacción física con objetos y personas, siendo uno de los medios más importantes de un robot para efectuar cambios en su entorno. Las tres principales funciones de una mano humana son explorar, sujetar y manipular objetos. La manipulación robótica se

centra en resolver los últimos dos, ya que el primero cae en el terreno de la háptica [10].

La interacción física presente en nuestra vida diaria cuando se realiza una manipulación, incluso en tareas simples y comunes, va más allá de tomar y colocar algún objeto. Ejemplos de estas tareas son encender la luz, extraer un libro de una estantería, abrir una puerta o cajón, empujar algún objeto, entre otros [11].

Justina es un robot de servicio diseñado en el Laboratorio de Bio-Robótica de la UNAM para operar en entornos domésticos, concebido como parte de una iniciativa para acercar la robótica a las personas y mejorar su calidad de vida [12]. Para resolver la manipulación cuenta con dos brazos de siete grados de libertad cada uno, lo que le permitiría transportar una mayor cantidad de objetos en un menor tiempo y llevar a cabo tareas más complejas.

Contar con dos brazos completamente funcionales abre la puerta a la integración de la manipulación colaborativa. Esto permitiría al robot manejar objetos de mayor tamaño, peso y complejidad, e incorporar acciones como la apertura de contenedores o la manipulación fina de objetos.

1.3. Planteamiento del problema

Los manipuladores de *Justina* exhiben deficiencias críticas que impidieron su participación en la RoboCup@HOME 2025. Los actuadores carecen de la capacidad de carga necesaria para las tareas de manipulación doméstica, los componentes estructurales presentan deformaciones permanentes por sobrecarga y el deterioro acumulado del sistema compromete su desempeño. Asimismo, aunque posee dos brazos, la manipulación colaborativa constituye un desafío técnico pendiente.

En respuesta a esta problemática, el presente trabajo plantea el rediseño e implementación integral de los subsistemas mecánico, electrónico y de software que conforman los brazos de *Justina*.

1.4. Alcance

El presente trabajo abarca el rediseño integral de los brazos manipuladores de *Justina* en tres disciplinas:

- **Mecánica:** diseño de manipuladores y efector finales, análisis por elemento finito y manufactura.
- **Electrónica:** arquitectura del sistema de control, integración de sensores y administración de energía.
- **Software:** desarrollo de nodos de control de los manipuladores y efectores finales e infraestructura para la manipulación colaborativa.

Se contempla además la validación experimental mediante pruebas de desempeño en tareas de manipulación simple en robótica de servicio.

1.5. Objetivos

Diseñar e implementar los subsistemas mecánico y electrónico de los brazos de un robot de servicio de propósito general con el fin de manipular objetos de uso común en un entorno doméstico, así como las interfaces para su integración en el *stack* de software.

1.5.1. Objetivos específicos

- Diseñar y manufacturar dos manipuladores robóticos.
- Diseñar y manufacturar dos efectores finales (EOAT) para manipular objetos comunes de uso doméstico.
- Implementar el nodo de control en ROS para resolver el modelo de la cinemática directa e inversa de la posición de los manipuladores.
- Implementar un nodo de control en ROS para tomar los objetos.
- Actualizar el paquete de ROS con la descripción del robot.

1.6. Organización del trabajo

A continuación se presenta una visión general de la estructura del presente trabajo, ofreciendo una breve descripción del contenido y propósito de cada uno de los capítulos que lo conforman.

En el Capítulo 2 se revisan los fundamentos teóricos y técnicos que sirven de base para el desarrollo del proyecto. Se abordan los temas de mecanismos y manipuladores robóticos, control de posición y trayectoria, medición de par y fuerza en las juntas, así como el uso del *Robot Operating System* (ROS) como plataforma de desarrollo.

En el Capítulo 3 se describe el proceso metodológico seguido para el diseño del sistema. Incluye el levantamiento de necesidades, la generación de especificaciones técnicas, el análisis comparativo de soluciones existentes mediante *benchmarking*, y el desarrollo del diseño conceptual. Asimismo, se detallan las herramientas utilizadas, como la matriz *Quality Function Deployment* (QFD), para la selección y evaluación de alternativas de diseño.

En el Capítulo 4 se presenta el desarrollo integral del sistema desde una perspectiva mecatrónica. El diseño mecánico comprende la estructura del manipulador y el efector final, además de los análisis por elemento finito y el proceso de manufactura. El diseño eléctrico abarca la arquitectura del sistema de control y la administración de energía. Finalmente, se detalla la instrumentación del software y la actualización de los nodos de control del robot *Justina*.

En el Capítulo 5 se describe el protocolo de validación del sistema mecánico y electrónico, así como las pruebas de manipulación implementadas para evaluar el desempeño del robot y sus resultados.

En el Capítulo 6 se presentan las conclusiones derivadas del trabajo, acompañadas de un análisis de los resultados y una discusión sobre los logros alcanzados. También se destacan las principales contribuciones del proyecto y se proponen posibles líneas de trabajo futuro que podrían continuar y mejorar el desarrollo realizado.

Capítulo 2

Antecedentes

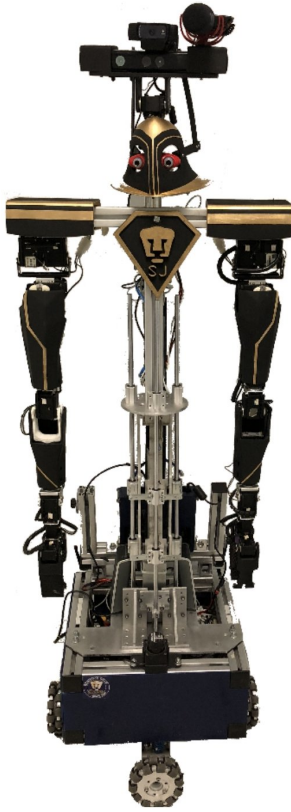
En la Universidad Nacional Autónoma de México (UNAM), el Laboratorio de Bio-Robótica de la Facultad de Ingeniería realiza investigación en áreas como robótica móvil, realidad virtual, reconocimiento de patrones, inteligencia artificial e interfaces hombre-máquina [13]. Fundado en 1996 por Jesús Savage Carmona, el laboratorio comenzó su trayectoria con la adquisición de un robot comercial estadounidense y, desde entonces, ha conformado equipos de estudiantes de licenciatura, maestría y doctorado para desarrollar prototipos propios, tales como TX8, TPR8, PAC-ITO y AL-ITA [14]. Cada plataforma ha sido fruto de una línea de trabajo orientada a la creación de robots de servicio para apoyar a las personas en sus actividades cotidianas en entornos como casas, escuelas, oficinas u hospitales [13].

Actualmente, el robot *Justina* se ha consolidado como la insignia del laboratorio y su referente en la competencia internacional RoboCup@Home. Al constituir el sistema de validación de este trabajo, en el presente capítulo se aborda su arquitectura de hardware y software, para posteriormente exponer la teoría general de manipuladores robóticos, en particular sus variables de interés, instrumentación y estrategias de control.

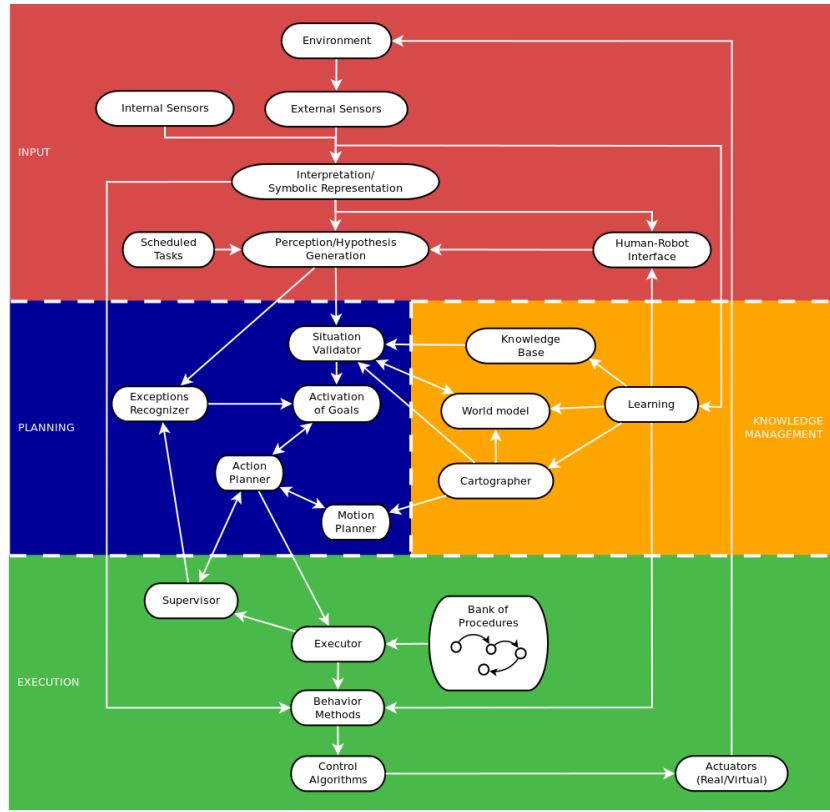
2.1. *Justina*

Justina (fig. 2.1a) es un robot de servicio doméstico cuyo diseño amigable e inteligente facilita la interacción con los humanos y su entorno. Como proyecto, se enmarca en una iniciativa para mejorar la calidad de vida de las personas, al tiempo que contribuye a la formación de estudiantes en ciencia e ingeniería [12]. La plataforma cuenta con una base móvil omnidireccional, un torso telescópico, un LiDAR, un micrófono cardioide, una cámara RGB-D y dos brazos manipuladores.

La integración del hardware con el software se fundamenta en la arquitectura ViRbot (fig. 2.1b), un sistema modular diseñado para coordinar la operación de robots de servicio autónomos combinando planeación de acciones con comportamientos reactivos sobre una representación dinámica del mundo [15]. Bajo este esquema, las funciones se organizan en cuatro capas: entrada, manejo del conocimiento, planeación y ejecución.



(a) Justina



(b) ViRbot

Figura 2.1: Arquitectura base del robot *Justina*

Para desempeñar sus tareas, un robot de servicio basado en la arquitectura ViRbot percibe el entorno mediante sus sensores, actualiza su representación del mundo, planifica sus acciones y transmite comandos de control a sus actuadores [16]. En la implementación actual de *Justina*, este modelo conceptual se materializa empleando *Robot Operating System* (ROS), un ecosistema de código abierto cuyo *framework* proporciona herramientas que facilitan la comunicación y sincronización de los procesos encargados del funcionamiento del robot [17].

En ROS, las responsabilidades del sistema se estructuran en *nodos*, unidades básicas de cómputo encargadas de realizar una función específica. Los nodos pueden comunicarse entre sí utilizando diversos mecanismos, lo que permite construir sistemas distribuidos en los que sus componentes pueden ejecutarse en distintas arquitecturas de hardware e incluso en diferentes sistemas operativos [18].

A medida que las necesidades de la comunidad superaron el enfoque inicial de ROS, la plataforma evolucionó hacia ROS 2 para contemplar, desde el diseño, nuevos casos de uso, como sistemas embebidos, control en tiempo real y aplicaciones industriales [19]. Para la interacción entre nodos, ROS 2 define tres interfaces de comunicación fundamentales [20]:

- *Tópicos*: Implementan un patrón publicador/suscriptor para la transmisión continua, unidireccional y asíncrona de datos, como lecturas de sensores o estados del sistema.
- *Servicios*: Implementan un patrón de solicitud/respuesta para interacciones síncronas de corta duración, donde un cliente requiere una respuesta o confirmación inmediata por parte de un servidor.
- *Acciones*: Implementan un patrón de meta/retroalimentación/resultado para tareas asíncronas de larga duración, donde un cliente puede enviar una meta, recibir retroalimentación periódica, cancelar la ejecución y obtener un resultado.

Estas primitivas de comunicación cimentan la arquitectura actual de *Justina*, permitiendo desacoplar el sistema en módulos orquestadores y módulos especializados, cuyos paquetes, nodos e interfaces se documentan detalladamente en el anexo A. Desde una perspectiva operativa, las tareas del robot provienen de rutinas preprogramadas, como las pruebas de la competencia RoboCup@Home, o de peticiones libres derivadas de comandos de voz. En ambos casos, el procesamiento de la solicitud (fig. 2.2) recae en los módulos orquestadores:

1. *Planning*: Descompone la petición en una secuencia coherente de acciones.
2. *Supervisor*: Examina cada paso del plan generado, proporcionando rutinas de recuperación ante cualquier imprevisto.
3. *Executor*: Traduce las instrucciones de alto nivel en llamadas a los métodos de los módulos especializados correspondientes.

A partir de este punto, la ejecución se distribuye en módulos especializados que proporcionan capacidades específicas:

- *Knowledge*: Administra el estado del entorno para consulta de los demás módulos.
- *HRI*: Canaliza la interacción con personas a través del reconocimiento y síntesis de voz.
- *Vision*: Procesa imágenes para la detección y clasificación de objetos y personas.
- *Navigation*: Localiza el robot y traza rutas hacia el destino evadiendo obstáculos.
- *Manipulation*: Genera y coordina las trayectorias de los brazos y el torso.
- *Hardware*: Abstrae la comunicación con sensores y actuadores.

Adicionalmente, mediante una red Ethernet, *Justina* distribuye el cómputo entre una computadora portátil, dedicada al comportamiento autónomo de alto nivel, y la computadora embebida en la base móvil, encargada del control de los componentes de bajo nivel.

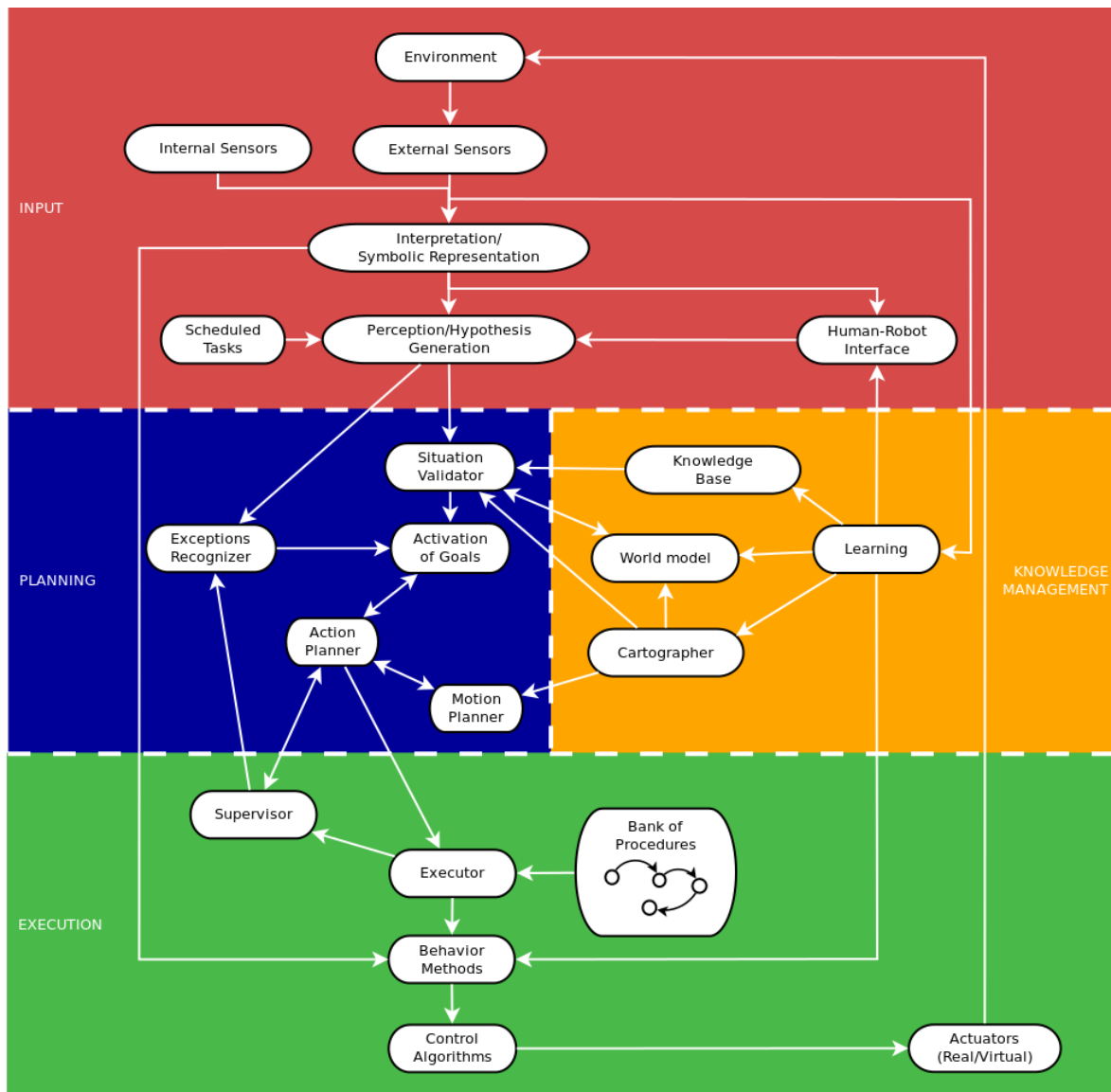


Figura 2.2: Arquitectura de operación de *Justina*.

2.2. Diseño de manipuladores robóticos

La ingeniería mecánica proporciona un conjunto de metodologías para definir y estudiar el movimiento en el diseño de manipuladores robóticos, y de máquinas en general, en situaciones estáticas y dinámicas. Las ramas predominantes encargadas de ese análisis son la cinemática y la dinámica [21]. La **cinemática** trata el estudio del movimiento sin considerar las fuerzas que lo ocasionan. La **dinámica**, por otro lado, se encarga del estudio de las fuerzas sobre sistemas en movimiento. Permite definir los esfuerzos a los que estarán sometidos los elementos del sistema, como función de la aceleración y las fuerzas inerciales [22].

2.2.1. Definición

Un **manipulador robótico** se define como una **cadena cinemática** que puede programarse para realizar una amplia variedad de tareas [21]. Una **cadena cinemática** es un conjunto de eslabones¹, conectados por pares cinemáticos², que producen movimientos relativos sincronizados con un propósito determinado [21].

Si cada eslabón en la cadena está conectado a al menos otros dos, la cadena cinemática forma uno o más bucles cerrados y se clasifica como una cadena cinemática **cerrada**. En caso contrario, se conoce como cadena cinemática **abierta** y constituye la base de los manipuladores robóticos seriales. Si la cadena consiste exclusivamente de eslabones binarios, se le conoce como **cerrada simple** [23].

2.2.2. Movilidad

La cinemática de los manipuladores robóticos se estudia como un mecanismo de varios grados de libertad [22]. Según Norton, un mecanismo es una cadena cinemática donde al menos uno de sus eslabones se ha “aterrizado” o fijado al sistema de referencia [22]. Reuleaux [24] limita su definición a cadenas cinemáticas cerradas y de un grado de libertad, a los cuales se les conoce como “restringidos”.

El número de grados de libertad globales de un manipulador determinan su **movilidad**. Es el número de variables de posición **independientes** que deben especificarse para conocer la posición de todos los eslabones de la cadena cinemática. Dicho de otra forma, es el número de entradas que se deben proporcionar para generar una salida predecible. Para determinarlo se debe de considerar el número de eslabones, el número de juntas y las interacciones entre ellos. La **condición de Gruebler** considera estos factores para un mecanismo plano resumidos en la ecuación 2.1 [22].

¹Cuerpo rígido con al menos dos nodos o puntos de conexión. Se clasifican en binarios, ternarios y cuaternarios, dependiendo del número de nodos que posean [22].

²Unión de dos o más eslabones en sus nodos, permitiendo movimiento relativo. Se clasifican por sus grados de libertad (GDL) y las más comunes son las juntas completas (1 GDL) y las semijuntas (2 GDL) [22].

$$M = 3L - 2J - 3G \quad (2.1)$$

donde:

M es el número de grados de libertad del mecanismo.

L es el número total de eslabones, incluido el eslabón fijo.

J es el número de pares cinemáticos.

G es el número de eslabones conectados a tierra.

Sin embargo, es importante destacar que cuando se tiene más de un eslabón conectado a tierra, el efecto que se obtiene es el de un eslabón más grande y de mayor orden. Por lo tanto, el término G en la ecuación de Gruebler 2.1 siempre será unitario. Además, dado que las semijuntas únicamente eliminan un grado de libertad en el plano, se consideran como $1/2$. El resultado es la **modificación de Kutzbach** mostrada en la ecuación 2.2.

$$M = 3(L - 1) - 2J_1 - J_2 \quad (2.2)$$

donde:

M es el número de grados de libertad del mecanismo.

L es el número total de eslabones, incluido el eslabón fijo.

J_1 es el número de pares cinemáticos de un grado de libertad.

J_2 es el número de pares cinemáticos de dos grados de libertad.

Finalmente, para mecanismos espaciales, la modificación de Kurtzbach se puede extender a eslabones con 6 GDL. De esta forma, se considera que las juntas completas anulan 5 GDL y las de orden inferior van restringiendo 1 GDL a la vez. Tomando en cuenta todos sus efectos se obtiene la ecuación 2.3, donde el subíndice indica el orden de cada tipo de junta.

$$M = 6(L - 1) - 5J_1 - 4J_2 - 3J_3 - 2J_4 - J_5 \quad (2.3)$$

2.2.3. Criterio de Grashof

Para el diseño de mecanismos de cuatro barras es necesario determinar qué configuraciones permiten obtener el movimiento requerido. En este sentido, la **ley de Grashof** establece que para que algún eslabón tenga rotación continua “la suma de las longitudes del eslabón más corto (s) y más largo (l) no puede ser más grande que la suma de las longitudes de los eslabones restantes (p y q)” [23], tal como se expresa en la inecuación 2.4.

$$s + l \leq p + q \quad (2.4)$$

Derivado de la condición de Grashof se puede agrupar a las cadenas cinemáticas de cuatro barras planas en tres clases, según se muestra en la figura 2.3. La **Clase I** cumple con la desigualdad 2.4 y sus inversiones³ serán la manivela-balancín, la doble manivela y un doble balancín de Grashof, donde el eslabón acoplador hace una revolución completa. La **Clase II** no cumple con la condición de Grashof y todas sus inversiones serán balancines triples. La **Clase III**, o caso especial de Grashof, es cuando se cumple la igualdad [22].

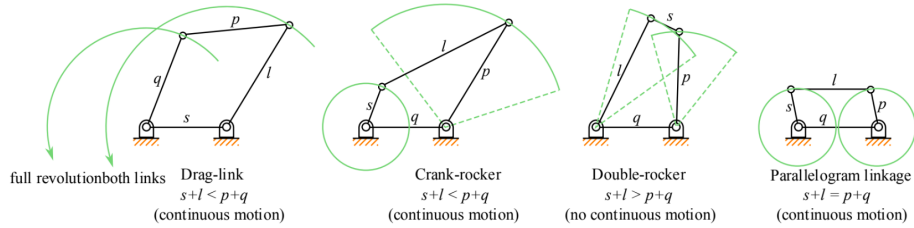


Figura 2.3: Casos del criterio de Grashof [25].

Este criterio constituye, por tanto, una condición preliminar para la síntesis dimensional del mecanismo.

2.2.4. Síntesis cinemática

La síntesis de un mecanismo no es una tarea trivial. Garantiza que, por ejemplo, un manipulador serial alcance los puntos de trabajo con las orientaciones deseadas del efector final [25]. Por esta razón, la síntesis cinemática se suele dividir en pasos: **síntesis de tipo o topológica, de número y dimensional**. Algunos autores suelen juntar la síntesis de tipo y de número en un solo paso, ya que la primera consiste en determinar el tipo de mecanismo que mejor resuelve el problema planteado, en un proceso puramente cualitativo basado en la experiencia y en la intuición [22], y el segundo consiste en determinar la cantidad de eslabones y tipo de juntas para obtener la movilidad requerida. El último paso está dedicado a definir las dimensiones del mecanismo [25].

De acuerdo con Norton [22], las dos grandes clases que engloban las técnicas existentes de síntesis dimensional son la gráfica o geométrica y la analítica. Los métodos gráficos se basan en principios básicos de geometría euclidiana y se centran en cadenas cinemáticas cerradas de cuatro barras. Varían en función de la cantidad de restricciones de diseño, tales como el número de posiciones, de pivotes móviles y fijos.

Una vez que se encuentre una solución potencial se debe evaluar su calidad. El proceso se vuelve entonces un **diseño cualitativo mediante análisis sucesivo** [22].

³Es la variante de una cadena cinemática dependiendo del eslabón que se designe como “fijo” o de referencia. Esto no cambia el movimiento relativo entre los eslabones, pero sí el movimiento absoluto generado [23].

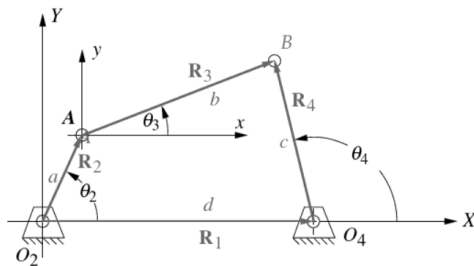
2.2.5. Análisis cinemático

El análisis cinemático permite determinar la configuración, velocidad y aceleración de los elementos móviles de un mecanismo a partir de sus relaciones geométricas. Así se garantiza que no fallará en condiciones de operación [22].

Al considerar el movimiento de un cuerpo rígido se debe partir de la postura o pose, que implica tanto la posición de un punto como la orientación de una línea sobre el eslabón [22], y se expresa con respecto a un marco de referencia⁴ asociado a otro eslabón [26]. De esta forma, la pose puede describirse mediante un componente traslacional, dado por el vector de posición ${}^j\mathbf{p}_i$, y uno rotacional, dado por la matriz ${}^j\mathbf{R}_i$. Con las transformaciones homogéneas, ambos efectos son combinados en una sola representación, como se muestra en la ecuación 2.5 [26]. Además de las matrices de rotación, la orientación puede representarse mediante cuaterniones [26]. En particular, esta representación es utilizada por ROS 2 para describir la orientación tridimensional de los marcos de referencia empleados en sus sistemas de coordenadas [27].

$${}^j\mathbf{T}_i = \begin{bmatrix} {}^j\mathbf{R}_i & {}^j\mathbf{p}_i \\ \mathbf{0}^T & 1 \end{bmatrix} \quad (2.5)$$

Para el análisis de posición de un mecanismo plano restringido, éste se representa mediante un lazo vectorial. Los eslabones se representan mediante vectores de posición, cuya suma es igual a cero, como se muestra en la figura 2.4. La notación compleja permite expandir estos vectores mediante la identidad de Euler. De esta forma, se obtiene un sistema de dos ecuaciones, con los ángulos θ_3 y θ_4 como incógnitas, como se expresa en la ecuación 2.6 [22]. Una vez obtenida la configuración del mecanismo, las ecuaciones de velocidad se obtienen mediante la diferenciación con respecto al tiempo de las ecuaciones de posición. Una segunda diferenciación permite obtener las ecuaciones de aceleración, utilizando los valores de posición y velocidad correspondientes al instante analizado.



$$\begin{cases} a \cos \theta_2 + b \cos \theta_3 - c \cos \theta_4 - d = 0 \\ a \sin \theta_2 + b \sin \theta_3 - c \sin \theta_4 = 0 \end{cases} \quad (2.6)$$

Figura 2.4: Lazo vectorial de posición de un mecanismo de 4 barras [22].

El mismo procedimiento puede aplicarse al análisis cinemático de un robot serial. Para un manipulador con n grados de libertad, cuando se conocen las variables articulares $\mathbf{q}$ y los parámetros geométricos de los eslabones, es posible resolver el problema

⁴Consiste de un origen O_i y una triada de vectores base $(\hat{x}_i, \hat{y}_i, \hat{z}_i)$.

de la **cinemática directa**, mediante el cual se determina la postura del efector final $\mathbf{x}$ con respecto al eslabón base. Esta relación puede expresarse mediante la matriz de transformación homogénea ${}^0\mathbf{T}_n$, resultado de la composición de las transformaciones asociadas a cada junta [26]. Por el contrario, cuando se conoce la pose deseada del efector final, el problema de la **cinemática inversa** permite determinar las variables articulares correspondientes [26].

A partir de la relación de cinemática directa, la diferenciación temporal permite establecer la relación entre las velocidades articulares y las velocidades lineal y angular del efector final [26]. Esta relación se expresa mediante el jacobiano, una transformación que relaciona el espacio de juntas con el espacio cartesiano [21], como se muestra en la ecuación 2.7. Una segunda diferenciación permite obtener la relación entre las aceleraciones articulares y las aceleraciones lineal y angular del efector final. Al derivar temporalmente la ecuación 2.7, se obtiene la ecuación 2.8 [28].

$$\dot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} \quad (2.7)$$

$$\ddot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\ddot{\mathbf{q}} + \dot{\mathbf{J}}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} \quad (2.8)$$

Las relaciones presentadas permiten establecer qué magnitudes deben conocerse durante la operación del sistema. En consecuencia, la selección de las variables de interés y de los elementos necesarios para su medición constituye el siguiente paso para llevar el modelo cinemático a una implementación física.

2.3. Variables de interés e instrumentación

2.4. Control de manipuladores robóticos

La teoría de control proporciona herramientas para diseñar y evaluar algoritmos para realizar los movimientos deseados o las aplicaciones de fuerza.

Capítulo 3

Metodología

Históricamente, el conocimiento práctico y la experiencia acumulada constituyeron la base del diseño. Como señalan Pahl y Beitz, a partir de la segunda mitad del siglo XX, la importancia económica del desarrollo eficiente y oportuno de productos motivó la creación de procedimientos sistemáticos y flexibles que permitieran planificar, optimizar, verificar y enseñar el proceso de diseño. Tal evolución propició la formalización de metodologías para analizar y descomponer problemas, generar soluciones óptimas y evaluar alternativas bajo criterios técnicos y económicos [29]. Estas metodologías, sin embargo, partían de especificaciones y requisitos que no consideraban a quienes usarían los productos.

El diseño centrado en el usuario (DCU) surgió en el laboratorio de Donald A. Norman en la Universidad de California San Diego como una filosofía que involucra al usuario en todas las fases del desarrollo de un producto, desde su conceptualización hasta su evaluación [30]. Norman propuso en *The Design of Everyday Things* un enfoque que parte de la comprensión profunda de las personas y de las necesidades que el diseño pretende satisfacer [31]. Este enfoque no es subjetivo, pues se fundamenta en disciplinas científicas como la ergonomía, la psicología y la antropología, y requiere que las decisiones de diseño estén respaldadas por datos obtenidos directamente de los usuarios [32].

En aplicaciones domésticas de robótica de servicio, el DCU resulta especialmente pertinente. Robots como Justina demuestran capacidades de navegación, manipulación, visión e interacción humano-robot que les permiten operar en ambientes reales no estandarizados, adaptándose a los hogares y a sus habitantes [33]. Identificar sistemáticamente las necesidades de quienes conviven con estos sistemas es, por lo tanto, una condición necesaria para orientar el desarrollo de soluciones robóticas apropiadas.

La filosofía del DCU establece que las decisiones de diseño deben fundamentarse en las necesidades reales de quienes interactúan con el producto. Este principio se adoptó en el rediseño de los manipuladores de Justina aplicando la metodología de Karl T. Ulrich y Steven D. Eppinger.

Ulrich y Eppinger presentan en *Diseño y Desarrollo de Productos* un conjunto de métodos para el diseño de productos físicos, discretos e ingenieriles [34]. Su proceso ge-

nérico se divide en seis fases: planeación, desarrollo del concepto, diseño a nivel sistema, diseño de detalle, pruebas y refinamiento, e inicio de producción.

Este capítulo abarca las fases de desarrollo del concepto y diseño a nivel sistema, que comprenden la identificación de necesidades, el establecimiento de especificaciones, la descomposición funcional, la generación de conceptos y la selección del concepto que se llevará al diseño de detalle.

3.1. Identificación de necesidades

La identificación de necesidades del cliente es la fase fundacional del proceso. Su objetivo es capturar de manera sistemática y exhaustiva las necesidades, expectativas y restricciones de los usuarios potenciales, transformando información cualitativa en un conjunto estructurado de requerimientos. Ulrich y Eppinger [34] definen una necesidad del cliente como una descripción, en las propias palabras del usuario, de los beneficios que un producto debe proporcionar, y distinguen con claridad entre necesidades —que describen *qué* debe lograr el producto— y soluciones —que especifican *cómo* lograrlo—. El proceso se estructura en cinco pasos: recopilación de datos sin procesar mediante entrevistas, interpretación de esos datos en términos de necesidades, organización jerárquica, establecimiento de importancia relativa y reflexión sobre los resultados.

3.1.1. Perfiles de usuario

Para la implementación del proceso de identificación de necesidades se definieron dos perfiles de participantes mediante muestreo intencional (Apéndice B).

El primero es el **usuario final**: personas que requieren asistencia doméstica constante debido a edad avanzada o alguna condición que limite su autonomía en actividades cotidianas del hogar, con capacidad cognitiva para participar en una entrevista y proporcionar consentimiento informado (Apéndice B). Sus necesidades se relacionan con la utilidad práctica del robot, la facilidad de interacción, la confiabilidad y la seguridad durante el uso.

El segundo perfil corresponde a **desarrolladores de software con experiencia en robótica de servicio**: miembros activos o previos del equipo de desarrollo de Justina con experiencia de participación en RoboCup@HOME y conocimiento técnico sobre el diseño, programación o implementación de funcionalidades del robot (Apéndice B). Este perfil aporta una perspectiva complementaria: han trabajado directamente con robots de servicio en condiciones desafiantes, han observado y comparado múltiples plataformas en competencia, y constituyen también un tipo de usuario del sistema al ser quienes realizan el mantenimiento y la programación del robot.

Dentro de este segundo perfil se distinguió entre quienes trabajan con **plataformas experimentales** de diseño propio y quienes utilizan **plataformas comerciales**, específicamente el *Toyota Human Support Robot* (Takeshi) del Laboratorio de Bio-Robótica (Apéndice B). Esta distinción motivó el diseño de variantes específicas en los

instrumentos de recolección.

3.1.2. Recopilación de datos

El proyecto contó con la aprobación del Comité de Ética en Investigación y Docencia de la Facultad de Ingeniería de la UNAM (Apéndice C). Todos los participantes firmaron una carta de consentimiento informado antes de su participación, bajo la cual se garantizó la confidencialidad de la información mediante codificación de datos, almacenamiento seguro y acceso restringido al equipo investigador (Apéndice C).

Siguiendo las recomendaciones de Ulrich y Eppinger [34], se seleccionaron las entrevistas como método principal de recopilación. Se diseñaron cuatro guiones semiestructurados diferenciados según perfil: uno para el usuario final, uno de aplicación general para desarrolladores, y dos variantes específicas para desarrolladores de plataformas experimentales y comerciales, respectivamente (Apéndice B). Los guiones completos se documentan en el Anexo [X].

Se realizaron nueve entrevistas en total: cinco con desarrolladores con experiencia en RoboCup@HOME y cuatro con usuarios finales potenciales. Todas fueron grabadas en audio con consentimiento explícito de los participantes (Apéndice C). Los fragmentos relevantes se transcribieron y organizaron en tablas estructuradas que vinculan cada declaración con su participante, el contexto de la pregunta y anotaciones sobre necesidades implícitas.

3.1.3. Interpretación y organización jerárquica

Siguiendo las directrices de Ulrich y Eppinger [34], cada necesidad se expresó como un atributo del producto —sujeto, verbo que describe la acción o capacidad deseada y, cuando es necesario, objeto o contexto de aplicación— capturando el *qué* sin anticipar el *cómo*. Se adoptó un enfoque holístico que abarcó el robot de servicio doméstico en su totalidad, reconociendo que las necesidades de manipulación no existen de manera aislada sino interrelacionadas con navegación, percepción e interacción. Como resultado, se identificaron **122 necesidades distintas**, mencionadas colectivamente **690 veces** a lo largo de todas las entrevistas.

Las 122 necesidades se organizaron en una jerarquía de **14 necesidades primarias** con sus correspondientes necesidades secundarias y terciarias. El agrupamiento se realizó en una hoja de cálculo de Excel, lo que permitió reorganizar iterativamente los enunciados y mantener trazabilidad entre cada necesidad y su fuente original. La estructura jerárquica completa se documenta en el Anexo [Y].

3.1.4. Importancia relativa y selección

Para determinar la importancia relativa de las 14 necesidades primarias se implementó una estrategia combinada. El primer componente fue la **frecuencia relativa de mención**: el cociente entre el número de menciones de cada necesidad primaria y sus

derivadas sobre el total de 690 menciones. El segundo componente fue una **encuesta estructurada** distribuida electrónicamente a tres de los participantes entrevistados, cuyos resultados se interpretaron como indicadores cualitativos de tendencia complementarios a los datos de frecuencia, dado el tamaño reducido de la muestra. La escala utilizada constó de cinco niveles: *Imprescindible* (1.0), *Deseable* (0.6), *Indiferente* (0.4), *Preferiría que no la tuviera* (0.2) y *No debería tenerla* (0.0). La importancia relativa final se calculó combinando el promedio de las evaluaciones directas con la frecuencia de mención de cada necesidad.

Aplicando un umbral de corte basado en la distribución de valores obtenidos, se seleccionaron seis necesidades primarias prioritarias. De estas, se retuvieron tres con implicación directa en el diseño de los manipuladores: *el robot manipula artículos domésticos*, *el robot es robusto* y *el robot tiene un diseño intuitivo*. A la primera se le incorporaron cuatro necesidades secundarias que detallan aspectos específicos de la capacidad de manipulación. El conjunto final quedó conformado por:

1. **El robot manipula artículos domésticos.**

- El robot planea trayectorias del manipulador.
- El robot toma artículos con la fuerza necesaria.
- El robot extiende el alcance efectivo de sus manipuladores.
- El robot manipula equipo para realizar tareas domésticas.

2. **El robot es robusto.**

3. **El robot tiene un diseño intuitivo.**

3.2. Establecimiento de especificaciones objetivo

La segunda fase de la metodología consiste en traducir las necesidades del cliente en especificaciones técnicas precisas y medibles. Ulrich y Eppinger [34] distinguen entre necesidades —expresadas en el lenguaje subjetivo del usuario— y especificaciones —que describen con exactitud cuantificable qué debe ser el producto—. Una especificación consiste en una *métrica* y un *valor* acompañado de sus unidades. Las especificaciones se establecen en dos momentos: primero como **especificaciones objetivo**, antes de conocer las restricciones técnicas del concepto de diseño, y luego como **especificaciones finales**, una vez seleccionado el concepto. El proceso comprende cuatro pasos: elaborar la lista de métricas, recabar información de referencia mediante *benchmarking*, establecer valores meta ideales y marginalmente aceptables, y reflexionar sobre los resultados.

3.2.1. Traducción de necesidades a métricas

Para cada necesidad del cliente se identificó la característica precisa y medible del producto que mejor refleja el grado en que esa necesidad queda satisfecha. La relación

entre necesidades y métricas se documentó en una matriz de correlación (Cuadro 3.2) que evidencia qué métricas cubren cada necesidad y permite identificar aquellas que requieren múltiples indicadores. A partir de las necesidades identificadas se definieron las ocho métricas presentadas en el Cuadro 3.1.

Cuadro 3.1: Lista de métricas derivadas de las necesidades del cliente.

#	Métrica	Nec. relacionadas	Importancia	Unidad
1	Capacidad de carga	1, 5, 6	5	[kg]
2	Fuerza de agarre	1, 3, 5	5	[N]
3	Alcance	2, 4, 5	5	[mm]
4	Apertura	1, 5	4	[mm]
5	Grados de libertad	4, 5	4	[1]
6	Repetibilidad	2, 5, 6	4	[mm]
7	Grado de protección IP	5, 6	3	[1]
8	Mapa de desensamble	6, 7	3	[s]

Cuadro 3.2: Matriz de correlación entre necesidades del cliente y métricas.

Necesidad	M1	M2	M3	M4	M5	M6	M7	M8
El robot manipula artículos domésticos	×	×		×				
El robot planea trayectorias del manipulador			×			×		
El robot toma artículos con la fuerza nec.		×						
El robot extiende el alcance efectivo			×		×			
El robot manipula equipo para tareas dom.	×	×	×	×	×	×	×	
El robot es robusto	×					×	×	×
El robot tiene un diseño intuitivo								×

3.2.2. Benchmarking y análisis competitivo

Para establecer valores de referencia se realizó un análisis comparativo de sistemas representativos de dos categorías: *cobots* de uso industrial —Panda/Franka Emika, Gen3/Kinova, UR3e/Universal Robots, EBLM/Ti5 Robot y OpenArm— y manipuladores de robots de servicio —Stretch 3/Hello Robot, HSR/Toyota, TIAGo/PAL Robotics y G1/Unitree—. Para los efectores finales se compararon grippers de dos dedos (Franka Hand, 2F-85 y 2F-140/Robotiq, HSR) y de tres dedos (3F-155/Robotiq, 3F-AR/UW-Milwaukee, DG-3F-B/Tesollo, Dex3s/Unitree).

La evaluación se realizó mediante un análisis de calidad de función (QFD) que ponderó cada métrica según su importancia relativa a las necesidades primarias identificadas. Para el brazo se evaluaron cuatro grupos de necesidades: planeación de trayectorias (alcance y repetibilidad), alcance efectivo (alcance y grados de libertad), tareas domésticas (capacidad de carga, alcance, grados de libertad, repetibilidad e IP) y robustez

(capacidad de carga, repetibilidad e IP). Para el gripper se evaluaron las necesidades de manipulación (capacidad de carga, fuerza de agarre, apertura, grados de libertad y repetibilidad), control de fuerza, tareas domésticas y robustez.

3.2.3. Definición de valores objetivo

Con base en el análisis de referencia se establecieron valores objetivo para cada métrica en dos niveles: el **valor ideal**, que representa el mejor desempeño razonablemente alcanzable, y el **valor marginal**, que constituye el mínimo aceptable para que el diseño sea viable. El Cuadro 3.3 presenta el conjunto completo de especificaciones objetivo.

Cuadro 3.3: Especificaciones objetivo para los manipuladores de Justina.

#	Métrica	Imp.	Unidad	V. marginal	V. ideal
1	Capacidad de carga	5	[kg]	> 2	> 3
2	Fuerza de agarre	5	[N]	> 70	> 100
3	Alcance	5	[mm]	> 500	> 700
4	Apertura	4	[mm]	> 100	> 140
5	Grados de libertad	4	[1]	≥ 5	≥ 6
6	Repetibilidad	4	[mm]	$< \pm 1$	$< \pm 0,1$
7	Grado de protección IP	3	[1]	IP44	IP55
8	Mapa de desensamble	3	[s]	—	—

La métrica M8 (mapa de desensamble), correspondiente a la necesidad de diseño intuitivo, se retiene en el conjunto como requerimiento cualitativo: su naturaleza no admite una cuantificación directa en esta etapa y su evaluación se formalizará durante la fase de selección de conceptos.

3.3. Generación de conceptos

La generación de conceptos es la actividad mediante la cual el equipo de desarrollo explora de manera sistemática el espacio de alternativas de diseño posibles antes de comprometerse con una solución particular. Ulrich y Eppinger [34] definen el concepto de un producto como una descripción aproximada de la tecnología, los principios de funcionamiento y la forma del producto. El método estructurado propuesto consta de cinco pasos: aclarar el problema mediante descomposición funcional, buscar externamente conceptos existentes, generar internamente nuevas soluciones, explorar sistemáticamente las combinaciones posibles y reflexionar sobre los resultados. Dado que la generación de conceptos es relativamente económica en comparación con las fases posteriores del desarrollo, los autores argumentan que no existe justificación para limitarla, y que un equipo eficiente debería explorar el espacio de alternativas con amplitud suficiente como para tener confianza en que no ha pasado por alto soluciones superiores.

3.3.1. Descomposición funcional del problema

El primer paso consistió en representar el sistema de manipulación como una caja negra que opera sobre flujos de energía, material y señales, para luego dividirla en subfunciones que describen lo que los elementos del manipulador deben hacer. De la totalidad de subfunciones identificadas se seleccionaron los **seis subproblemas críticos** que determinan en mayor medida el desempeño del sistema: aceptar energía, convertir energía a movimiento, transmitir movimiento y fuerza, sensor posición, sensor fuerza y sostener el objeto. El diagrama funcional completo, que muestra la interacción entre estas subfunciones a través de los flujos de energía y señal, se presenta en la Figura

3.3.2. Búsqueda externa de soluciones

Para cada uno de los seis subproblemas críticos se realizó una búsqueda exhaustiva de soluciones existentes en la literatura técnica, catálogos de fabricantes, patentes y en el análisis directo de los robots estudiados durante el benchmarking. Las soluciones encontradas se organizaron en tablas morfológicas —una por subproblema— que cubren los principales principios de solución para cada función. Las tablas completas se incluyen en el Anexo [W].

3.3.3. Exploración sistemática y acotamiento del espacio de soluciones

La combinación irrestricta de las soluciones identificadas genera un espacio de diseño de gran magnitud. Mediante un script de Python se enumeraron sistemáticamente **1943 combinaciones** que representan el espacio de búsqueda completo antes de aplicar criterios de compatibilidad.

Para reducir este espacio a candidatos viables se procedió en dos etapas. Primero, se acotó el conjunto de opciones por subproblema a las soluciones más pertinentes para el contexto de aplicación: robot de servicio doméstico, operación eléctrica e integración con el stack de software existente del laboratorio. Segundo, sobre este espacio reducido se aplicó un criterio de **afinidad entre soluciones**: se conservaron únicamente aquellas combinaciones que representan configuraciones lógicamente coherentes y directamente implementables, que no requieren adaptaciones innecesarias para funcionar conjuntamente. Este filtrado redujo el espacio de candidatos a **384 combinaciones realistas**.

3.3.4. Desarrollo de conceptos completos

Del conjunto de 384 candidatos se seleccionaron **4 conceptos representativos** para su desarrollo en detalle, buscando cubrir la diversidad de enfoques presentes en el espacio de soluciones. Cada concepto se materializó en un modelo CAD conceptual que permitió evaluar de manera preliminar las métricas establecidas en las especificaciones

objetivo y facilitar la comparación estructurada en la fase de selección. Los cuatro conceptos, sus características principales y su evaluación frente a las especificaciones se describen en el capítulo siguiente.

Apéndice A

Justina

La arquitectura de software del robot Justina se encuentra organizada en diferentes módulos funcionales. Cada módulo agrupa los paquetes de software que implementan una determinada función del sistema. En esta sección se describe la estructura de cada módulo, sus paquetes, nodos y las interfaces de comunicación.

A.0.1. Executor

El módulo *Executor* se encarga de ejecutar, paso a paso, el plan generado por *Planning/Supervisor*: solicita cada instrucción, la traduce a una meta de acción concreta y despacha su ejecución a través de una colección de máquinas de estado (FSM) que envuelven las capacidades del robot (navegación, manipulación, HRI, visión y hardware), reportando el resultado de vuelta al Supervisor.

Paquetes

- `executor_manager`

Puente entre el *Supervisor* (planificador) y el *Executor Worker*. Solicita la siguiente instrucción del plan, convierte el comando de texto crudo (p. ej. "*Justina goto table*") en una meta `TaskStep/ExecuteTask`, la despacha al Worker y gestiona el ciclo de vida de la tarea: pide el siguiente paso si tuvo éxito, salta al siguiente si fue rechazada, y reintenta (hasta `MAX_RETRIES`) si falló.

- `executor_worker`

Capa de ejecución física: implementa un servidor de acciones de ROS 2 que recibe las metas del `executor_manager` y las despacha, mediante un `actionMap` de cadenas a *lambdas*, a la clase FSM correspondiente de `executor_tools`. Sigue una política *accept-to-report*: incluso las acciones inválidas se aceptan para poder reportar formalmente el rechazo al Supervisor. Cada tarea aceptada corre en un hilo separado para no bloquear el executor principal del nodo.

■ `executor_tools`

Librería de clases FSM, una por cada primitiva de acción del robot; cada una encapsula un comportamiento concreto y se comunica con los módulos de Navigation, Manipulation, HRI, Knowledge, Vision y Hardware a través de sus respectivas librerías `*_tools`. Todas devuelven un `Result` uniforme (`success`, `state`, `message`). Las primitivas registradas en el `actionMap` de `executor_worker_node` son:

- `say` (`Say`) – Pronuncia por TTS una frase obtenida del banco de conocimiento a partir de una clave.
- `listen` (`Listen`) – Captura audio, extrae una entidad nombrada y, si aplica, la persiste en el estado del mundo.
- `go_to` (`GoTo`) – Navega a una ubicación conocida, ajustando primero la altura del torso.
- `check` (`Check`) – Verifica el estado de un sensor o componente de hardware (p. ej. una puerta).
- `point` (`Point`) – Alinea base y cabeza hacia un objetivo y extiende el brazo para señalarlo.
- `give` (`Give`) – Selecciona la mano que sostiene el objeto solicitado, la extiende hacia la persona y abre el gripper al confirmar la entrega.
- `find_person` (`FindPerson`) – Escanea con la cabeza, detecta y localiza a una persona, la identifica o registra, y actualiza su descripción en el estado del mundo.
- `align` (`Align`) – Alinea la base frente a un objetivo visual con control de lazo cerrado y verificación de estabilidad por muestras consecutivas.
- `drop` (`Drop`) – Consume una TF de espacio seguro generada por `FindSpace` y ejecuta la maniobra inversa de `Take`: coloca el objeto y libera el gripper.
- `find_space` (`FindSpace`) – Encuentra y valida una TF de espacio libre (mesa, repisa o contenedor) para depositar un objeto, sin mover brazos ni soltar nada.
- `find_object` (`FindObject`) – Mueve la cabeza, detecta un objeto por visión y actualiza su pose y ubicación en el estado del mundo.
- `focus` (`Focus`) – Centra la cámara/cabeza en un objetivo detectado previamente.
- `receive` (`Receive`) – Verifica que la mano esté libre, extiende el brazo, abre el gripper y lo cierra al recibir un objeto de una persona.
- `follow` (`Follow`) – Activa o desactiva el servicio de seguimiento de personas de HRI.

- **approach / approach_partial (Approach)** – Acerca al robot a una persona u objeto navegando a una pose calculada; **approach_partial** invoca una variante parcial (**runPartial**) para aproximaciones controladas/incompletas.
- **take (Take)** – FSM principal para tomar un objeto: obtiene el objetivo, planea y ejecuta el pregrasp apoyándose en **VisualGraspApproach** y en **manipulation_tools**, cierra el gripper y retrae la trayectoria.
- **verify_take (VerifyTake)** – Confirma por visión si el objeto sigue presente tras un intento de toma y limpia el estado de "mano ocupada" si ya no lo está.
- **find_pose (FindPose)** – Detecta y localiza la pose de una persona (p. ej. el cliente) y la guarda en el estado del mundo y en un buffer interno compartido.
- **analyze_objects (AnalyzeObjects)** – Ejecuta análisis de conteo, tamaño, color o comparación sobre objetos detectados, actualizando la analítica en el estado del mundo.
- **analyze_person (AnalyzePerson)** – Analiza color de prendas y/o gestos de una persona detectada, actualizando la analítica correspondiente.
- **save_loc (SaveLoc)** – Guarda la pose actual (o una calculada tras avanzar/rotar) como una ubicación nombrada.
- **fold (Fold)** – Dobla una prenda plana sobre una mesa en dos fases (mangas y luego base), detectando puntos clave por visión.

Además, **VisualGraspApproach** es una clase de soporte (no registrada como primitiva propia) que **Take** usa para calcular y planear la aproximación fina de agarre: candidatos **FRONT/ TOP**, corrección visual por el TF del marcador del gripper, y cálculo del retroceso simétrico al avance real de la base.

■ **gui_controller**

Herramienta de escritorio (PyQt5) para desarrollo y pruebas: lanza, detiene y monitorea los procesos/nodos ROS 2 de todo el sistema de Justina a partir de un catálogo de comandos (**config.yaml**) y una lista de nodos esperados (**nodes.yaml**). No forma parte del grafo de ejecución en tiempo de competencia.

Nodos

■ **executor_manager_node**

Orquesta el flujo de ejecución del plan: al recibir un disparo, solicita la siguiente instrucción al Supervisor, la parsea a una meta de acción y la envía al Executor Worker, reaccionando a los estados **SUCCEEDED**, **REJECTED**, **FAILURE** y **ABORTED** con lógica de siguiente paso, salto o reintento.

- **Tópicos suscritos:**

- `/input_trigger` – Dispara el inicio (o continuación) del ciclo de solicitud y ejecución del plan.

- **Servicios utilizados:**

- `/request_instruction` – Solicita al Supervisor el siguiente paso del plan.
- `/supervisor/supervisor_executor/authorize` – Reporta al Supervisor la autorización/resultado asociado a la tarea en curso.

- **Acciones utilizadas:**

- `/executor/execute_task` – Envía cada paso parseado como meta de acción al Executor Worker y monitorea su resultado, con reintento configurable ante fallas.

- **executor_worker_node**

Servidor de acciones que recibe cada meta de tarea, la despacha (según el campo `action`) a la clase FSM correspondiente de `executor_tools`, la ejecuta en un hilo separado y reporta el resultado. No crea tópicos ni servicios propios directamente: los clientes concretos de Navigation, Manipulation, HRI, Knowledge, Vision y Hardware viven dentro de cada clase FSM y ya están documentados en las secciones de sus respectivos módulos; se listan de forma compacta en el `actionMap` descrito en *Paquetes* para evitar duplicarlos aquí.

- **Acciones ofrecidas:**

- `/executor/execute_task` – Acepta toda meta recibida (política *accept-to-report*), despacha a la herramienta correspondiente y reporta SUCCESS/FAILURE/REJECTED/ABORTED.

- **gui_controller**

Nodo ROS 2 mínimo (`ROS2NodeChecker`) que respalda al panel de control de escritorio: enumera los nodos activos del sistema (vía la API de `ros2node`) y permite finalizarlos por nombre (`pkill`) para reiniciar partes del sistema durante pruebas. No crea tópicos, servicios ni acciones propias: introspecciona y detiene nodos por fuera de ROS 2 (API de listado de nodos y señales de sistema).

A.0.2. HRI

El módulo *HRI* (*Human-Robot Interaction*) se encarga de la interacción hablada con las personas (síntesis y reconocimiento de voz), de la detección y seguimiento de personas, y de la interfaz visual de operación del robot. Localiza a la persona objetivo combinando LiDAR (piernas) y visión (cuerpo), mantiene al robot orientado o navegando hacia ella cuando el seguimiento está activo, y expone estas capacidades al resto del sistema a través de la librería cliente `hri_tools`. A diferencia del resto del módulo, el paquete `gui` corre físicamente en la computadora embebida en la base Robotino

4 (repositorio `src_Robotino`), junto con la capa de *Hardware* de bajo nivel; se documenta aquí porque el propio árbol de ese repositorio lo clasifica bajo HRI, y porque funcionalmente es un panel de operación que consume tópicos de HRI y Vision.

Paquetes

- `hri_tools`

Librería *Facade* (clase `HRI`) que envuelve como llamadas síncronas los servicios de `Talker`, `Listener`, seguimiento de personas y detección de gestos, para que otros módulos (p. ej. `Say` y `Listen` de `executor_tools`) no manejen directamente clientes asíncronos de ROS 2. También aloja el *launch* raíz del módulo (`hri.launch.xml`), que agrupa bajo el *namespace* `/hri` a `Talker`, `Listener`, `human_detector` y `human_follower`.

- `human_detector` (paquete ROS `human_detection`)

Colección de nodos Python basados en YOLO para detección de personas. En el sistema activo, `hri.launch.xml` sólo lanza `person_tracker`, que fusiona detecciones YOLO con la nube de puntos de la cámara para estimar la posición 3D de la persona objetivo, mueve la cabeza hacia ella y expone el enfoque de visión como servicio. `pose_detector.py` – el servidor de `/human_detection/detect_wrist` que consume `HRI::FindPose` – **sí corre en el sistema activo**, pero se lanza desde el módulo *Vision* (`vision_tools/launch/vision.launch.xml`, como `pose_detector_node`), no desde `hri.launch.xml`; se corrige aquí una afirmación anterior de este documento que lo daba por no lanzado. El resto de *entry points* del paquete (`detector`, `webcam_image`, `person_tracker_pose`, `pointing`, `pointing_v2`, `client`, `object_description`, `face_detect`) sí son scripts de desarrollo/prototipo sin *launch* activo.

- `human_follower`

Pipeline de seguimiento de personas: `leg_finder_node` detecta piernas por LiDAR; `human_follower_nodeV2` fusiona esa detección con la posición de cuerpo de `person_tracker` mediante un filtro de Kalman y publica una meta de navegación hacia la persona; `human_tracking_node` expone el interruptor de alto nivel del seguimiento con vigilancia por *watchdog*. El paquete también compila las variantes `leg_finder_nodev2` y `human_follower_node` (v1), que no están conectadas al *launch* activo.

- `listener`

Reconocimiento de voz: captura audio del micrófono, aplica detección de actividad de voz (VAD) y transcribe con Whisper (con soporte de gramáticas JSON por escenario), exponiendo el resultado como servicio.

- `talker`

Síntesis de voz (TTS, motor Piper) expuesta como servicio.

- `gui` (paquete ROS `justina_gui`; repositorio `src_Robotino`, corre en la computadora de la base Robotino 4)

Panel de operación (PyQt, clase `HSRBGui`) que muestra en una pantalla el estado del robot durante la competencia: estado de *run-stop*, el texto que Listener/Talker están procesando, y las imágenes de depuración de los detectores de Vision/HRI (objetos, manos saludando, rostros, asientos vacíos). Es puramente un suscriptor/visualizador; no publica tópicos ni ofrece servicios propios. Se lanza junto con los nodos de *Hardware* de bajo nivel desde `justina_hardware`, bajo el *namespace* `/hardware` (no `/hri`).

Nodos

- `person_tracker`

Fusiona las detecciones de YOLO con la nube de puntos de la cámara para estimar y publicar la posición 3D de la persona objetivo, mover la cabeza hacia ella y exponer el enfoque de visión como servicio.

- **Tópicos publicados:**

- `/human_detection/target_point` – Posición 3D estimada de la persona objetivo.
- `/hri/human_tracking/confidence` – Confianza de la estimación actual.
- `/hardware/head/look_at` – Punto al que debe mirar la cabeza para centrar a la persona.
- `/human_detection/track_markers` – Marcadores de depuración del seguimiento (RViz).
- `/navigation/camera_obstacles/bypass` – Indica a navegación que ignore temporalmente los obstáculos de cámara.

- **Tópicos suscritos:**

- `/vision/detections_list` – Lista de detecciones YOLO.
- `/hardware/camera/points` – Nube de puntos de la cámara (parámetro `point_cloud_topic`).

- **Servicios ofrecidos:**

- `/vision/focus` – Activa/desactiva el enfoque continuo en la persona detectada.
- `/vision/focus/enable` / `/vision/focus/disable` – Variantes tipo *Trigger* para activar/desactivar el enfoque.

- **Servicios utilizados:**

- `/vision/enable_continuous` – Habilita la detección continua en el nodo de visión.

- `pose_detector_node` (paquete `human_detection`; lanzado desde `Vision/vision_tools/launch/` no desde `hri.launch.xml` – ver nota en *Paquetes*)

Detecta si una persona está saludando/apuntando con la muñeca y en qué dirección, a partir de la imagen y la nube de puntos de la cámara.

- **Tópicos publicados:**

- `/human_detection/pose_image_wrist` – Imagen de depuración de la detección (consumida por `gui_node`).

- **Tópicos suscritos:**

- Imagen RGB, nube de puntos y `camera_info` de la cámara (parámetros `image_topic/pointcloud_topic`).

- **Servicios ofrecidos:**

- `/human_detection/detect_wrist` – Detecta si hay una persona haciendo un gesto de muñeca/saludo y en qué dirección (usado por `HRI::FindPose`).

- `leg_finder_node`

Procesa el láser de la base para detectar patrones de piernas humanas y publica la posición de la más probable frente al robot.

- **Tópicos publicados:**

- `/hri/leg_finder/legs_pose` – Posición estimada de las piernas detectadas.
- `/hri/leg_finder/marker` – Marcador de depuración (RViz).

- **Tópicos suscritos:**

- `/hardware/base/scan` – Barrido láser de la base (parámetro `scan_topic`).
- `/hri/leg_finder/enable` – Activa/desactiva la detección.

- `human_follower_nodeV2`

Fusiona (filtro de Kalman de 4 estados) la posición de piernas y de cuerpo de la persona objetivo, y publica una meta de navegación para mantener una distancia fija hacia ella mientras el seguimiento está activo.

- **Tópicos publicados:**

- `/nav_control/goal` – Meta de navegación hacia la persona (parámetro `goal_topic`).
- `/navigation/stop` – Señal de detención cuando se pierde el objetivo (parámetro `stop_topic`).

- `/hri/human_follower/state` – Estado textual del seguimiento.
- `/hri/human_follower/debug_fused_point` – Punto fusionado de depuración.

- **Tópicos suscritos:**

- `/hri/human_following/enable` – Activa/desactiva el seguimiento.
- `/hri/leg_finder/legs_pose` – Posición de piernas detectada por `leg_finder_node`.
- `/human_detection/target_point` – Posición de cuerpo estimada por `person_tracker`.

- **human_tracking_node**

Expone el interruptor de alto nivel del seguimiento de personas: al activarse, habilita detección continua de visión, detección de piernas y seguimiento de cuerpo; vigila la confianza reportada con un *watchdog* para desactivarse automáticamente si se pierde el objetivo.

- **Tópicos publicados:**

- `/hri/human_following/enable` – Habilita/deshabilita `human_follower_nodeV2`.
- `/hri/leg_finder/enable` – Habilita/deshabilita `leg_finder_node`.
- `/vision/enable_continuous` – Habilita/deshabilita la detección continua de visión.
- `/hri/human_tracking/state` – Estado agregado del seguimiento.

- **Tópicos suscritos:**

- `/hri/human_tracking/confidence` – Confianza reportada por `person_tracker`, usada por el *watchdog*.

- **Servicios ofrecidos:**

- `/hri/human_tracking/enable` – Activa/desactiva el seguimiento de personas de extremo a extremo (usado por la primitiva `follow` del Ejecutor).

- **Servicios utilizados:**

- `/vision/focus/enable` / `/vision/focus/disable` – Activa/desactiva el enfoque de visión en la persona objetivo.

- **listener_node**

Captura audio local del micrófono (no un tópico ROS), aplica detección de actividad de voz y transcribe con Whisper (opcionalmente restringido a una gramática JSON por escenario), devolviendo el texto reconocido de forma síncrona.

- **Tópicos publicados:**

- `/hsrb/gui/state` – Estado del reconocimiento para el GUI de operación.

- **Servicios ofrecidos:**

- `/hri/listener/listen` – Ejecuta un ciclo de reconocimiento de voz y regresa el texto transcrito. (El parámetro de *override* del nombre de servicio expuesto por el *launch*, `listen_service`, no coincide con el parámetro que el nodo realmente declara, `listener_service`, por lo que en la práctica siempre se usa el valor por defecto `listen`, resuelto bajo el *namespace* del nodo.)

- **talker_node**

Sintetiza voz a partir de texto usando el motor Piper y la reproduce por el altavoz del robot.

- **Tópicos publicados:**

- `/hsrb/gui/text` – Texto en curso de síntesis, para el GUI de operación.

- **Servicios ofrecidos:**

- `/hri/talker/talk` – Sintetiza y reproduce el texto recibido; regresa éxito/fracaso.

- **gui_node** (nombre interno del nodo: `justina_gui_node`; corre en la computadora de la base Robotino 4, bajo el *namespace* `/hardware`)

Panel de operación en pantalla: refleja el estado de *run-stop*, el texto de Listener/Talker, y las imágenes de depuración de los detectores de personas/objetos/rostros/asientos para que el operador supervise a Justina durante una prueba o competencia.

- **Tópicos suscritos:**

- `/navigation/navigation_ctrl/stop` – Estado del paro de emergencia/*run-stop*.
- `/hsrb/gui/state` – Estado textual publicado por `listener_node`.
- `/hsrb/gui/text` – Texto en curso de síntesis publicado por `talker_node`.
- `/vision/owl/debug_image` – Imagen de depuración del detector de objetos (OWL) de Vision.
- `/human_detection/pose_image_wrist` – Imagen de depuración de detección de saludo/muñeca.
- `/face_recog/debug_image` – Imagen de depuración del reconocimiento facial de Vision.
- `/vision/detect_seats/debug_image` – Imagen de depuración de detección de asientos vacíos de Vision.

A.0.3. Hardware

El módulo *Hardware* se encarga de exponer el hardware físico del robot (cabeza, torso, brazos, pinzas, base omnidireccional, cámara RGB-D y LiDAR) como una fachada de ROS 2 síncrona para el resto del sistema, y de controlar cada uno de esos actuadores/sensores a bajo nivel.

Arquitectura distribuida: el sistema de Justina corre en dos computadoras que forman un mismo grafo de ROS 2 (descubrimiento DDS/CycloneDDS sobre una red Ethernet dedicada), ambas bajo el *namespace* común `/hardware`:

- La **computadora principal** (repositorio `src.zip`, ya documentado arriba) ejecuta `hardware_tools` (la fachada `Hardware`), `door_detector`, `scanner` (puente de orientación de cabeza), `kinect_ros2` y el *driver* de cámara RGB-D (`openni2_camera`), además de todos los módulos de software de más alto nivel (Executor, HRI, Knowledge, Manipulation, Navigation, Planning, Supervisor, Vision).
- La **computadora embebida en la base Robotino 4 (FESTO)** (repositorio `src_Robotino`) ejecuta los *drivers* de bajo nivel de la base, cabeza, torso, brazos y pinzas (`justina_hardware`, `base`, `head`, `torso`, `arms`, `grippers`, `joint_states_parser`) y el panel de operación (`gui`, documentado en la sección de *HRI* por la categorización del propio repositorio).

El paquete `justina_description` está duplicado byte a byte en ambos repositorios (cada computadora necesita el URDF para su propio `robot_state_publisher` / TF local), por lo que se documenta una sola vez. Los dos repositorios son consistentes entre sí: los nombres de tópicos y servicios que `hardware_tools/Hardware.hpp` espera del lado de la base Robotino (p. ej. `/hardware/arms/move_right_arm`, `/hardware/torso/current_pose`) coinciden exactamente con los que los nodos de `src_Robotino` realmente exponen.

Paquetes

- `hardware_tools`

Fachada síncrona (clase `Hardware`) sobre los servicios de torso, cabeza, brazos, pinzas, base y detección de puerta, para que otros módulos no manejen clientes asíncronos de ROS 2 directamente. También aloja el *launch* raíz del módulo (`hardware.launch.xml`), que bajo el *namespace* `/hardware` levanta el *driver* de cámara RGB-D (`openni2_camera`) y la generación de nube de puntos (`depth_image_proc`) como nodos componibles, además de `door_detector` y `scanner`; y, condicionalmente, `takeshi_hardware.launch.xml` – el *bringup* alternativo para el backend de hardware del robot Takeshi (HSR), no relacionado con Justina.

- `door_detector`

Determina si una puerta está abierta o cerrada comparando la distancia promedio del sector central del LiDAR contra un umbral, y expone el resultado como servicio.

- **scanner** (paquete con nombre heredado; el nodo que contiene es en realidad un puente de orientación de cabeza, no un controlador de LiDAR)

Traduce un punto 3D de interés (publicado por `Hardware::LookAt`) en un objetivo de cabeza (pan/tilt), calculando el ángulo necesario vía TF entre el marco de la cámara/cuello y el objetivo, respetando límites articulares y una zona muerta para evitar micro-movimientos.

- **kinect_ros2**

Controlador de terceros (basado en `libfreenect`) para la cámara Kinect v1, vendorizado en el árbol de código. No forma parte del *pipeline* activo de cámara (que usa `openni2_camera`); sólo se referencia desde *launches* de depuración.

- **justina_description**

Modelo del robot (URDF, mallas 3D) y entorno de simulación Webots. Además del *launch* de *bringup* para el robot real (`display.launch.xml`, que publica el URDF vía `robot_state_publisher` e incluye los *drivers* de cabeza/brazos/torso/pinzas fuera de este repositorio), aloja tres nodos auxiliares con código fuente propio:

- `simple_justina_sim.py` – Simulador ligero en software: integra `/hardware/base/cmd_vel`, publica odometría, TF `odom→base_link`, articulaciones falsas y un láser simulado (contra el mapa de navegación si está disponible); además implementa directamente los servicios `move_head/move_torso/move_arms` para sustituir a los *drivers* reales durante pruebas sin robot físico (usado por `justina_sim.launch.xml`).
- `kinect_tilt_motor_node.py` – Refleja el *tilt* de cabeza comandado hacia el motor físico de inclinación del Kinect v1 vía USB; sólo se lanza cuando `use_kinect` está activo.
- `virtual_pointcloud.cpp (depth_to_pointcloud)` – Convierte una imagen de profundidad y su `camera_info` en una nube de puntos; utilidad de depuración, no forma parte del *pipeline* principal de percepción.

- **justina_hardware** (repositorio `src_Robotino`)

Bringup raíz del lado Robotino: bajo el *namespace* `/hardware`, remapea los tópicos `joint_states` relativos de cada driver (`base`, `head`, `arms/left_arm`, `arms/right_arm`, `torso`, `grippers/left_gripper`, `grippers/right_gripper`) al tópico canónico `/hardware/joint_states`; incluye los *launches* de `head`, `arms`, `torso`, `grippers` y `joint_states_parser`; lanza el `gui_node` del panel de operación; y levanta un segundo `robot_state_publisher` (con el mismo URDF que el de la computadora principal) cuyo tópico de entrada de articulaciones se remapea a `/manipulation/joint_states` – la salida ya fusionada de `joint_states_parser`.

- **base** (repositorio `src_Robotino`)

Controlador de la base Robotino 4: envuelve el cliente TCP hacia el servidor *RobotinoAPI2* del fabricante (clase *ComROS*) para exponer como ROS 2 tracción omnidireccional, sensor de choque (*bumper*), lecturas de motores, gestión de energía, IMU, entradas/salidas digitales y encoders (patrón adaptado del *driver* comunitario *robotino_node*). También construye, como ejecutables independientes: *robotino_odometry_node* (odometría + TF *odom→base_link*, separado del nodo principal) y *Stop_node* (traduce una entrada digital física de paro de emergencia a la señal de alto de navegación). Los ejecutables *robotino_camera_node* y *robotino_laserrangefinder_node* se compilan pero no se incluyen en el *launch* activo (la cámara y el LiDAR se sirven en su lugar por *openni2_camera*, en la computadora principal, y por el paquete de terceros *urg_node*, respectivamente). El paquete también lanza *urg_node* (*driver* de terceros para el LiDAR Hokuyo/URG usado por Justina).

- **arms** (repositorio *src_Robotino*)

Controla los 6 servomotores Dynamixel de cada brazo (hombro doble, brazo, codo, muñeca – según los *joint_names* configurados) vía el SDK de Dynamixel sobre RS-485. El mismo ejecutable (*arms_node*) se lanza dos veces, una por lado, parametrizado por *config/left_arm.yaml* / *right_arm.yaml* (puerto serial, IDs de servo, límites, dirección de giro). Incluye una compensación de gravedad simple: un arreglo fijo de *offsets* por articulación (*compensation_bits*) sumado al comando antes de escribirlo al motor, activable con *compensation_enable*.

- **head** (repositorio *src_Robotino*)

Controla los 2 servomotores Dynamixel de pan/tilt de la cabeza; misma arquitectura de nodo que *arms* (servicio de movimiento, tópicos de articulaciones/pose actual/objetivo, parada por */navigation/navigation_ctrl/stop*), sin compensación de gravedad.

- **torso** (repositorio *src_Robotino*)

Controla el eje lineal del torso mediante un controlador de motor Jrk (Pololu) sobre puerto serial (*/dev/torso*), usando retroalimentación analógica de posición con tolerancia y un número mínimo de lecturas estables antes de considerar la meta alcanzada.

- **grippers** (repositorio *src_Robotino*)

Controla las pinzas mediante un microcontrolador embebido propio (RP2350 + módulo Ethernet W5500, protocolo binario de tramas fijas de 8 bytes, librería *gripper_sdk* vendorizada en el mismo paquete) conectado por TCP/IP, en vez de un bus serial compartido como brazos/cabeza/torso. El mismo ejecutable (*grippers_node*) se lanza dos veces (una por lado), parametrizado por *config/left_gripper.yaml* / *right_gripper.yaml* (IP, puerto y nombres de articulación de cada pinza).

- `joint_states_parser` (repositorio `src_Robotino`)

Fusiona los mensajes `JointState` parciales que cada *driver* (base, brazos, cabeza, pinzas) publica bajo `/hardware/joint_states` en un único mensaje completo, con todas las articulaciones del robot en un orden canónico fijo, para consumo de MoveIt2 (lo publica en `/manipulation/joint_states`).

Nodos

- `door_detector_node`

Analiza el sector central del barrido láser de la base para estimar si la puerta frente al robot está abierta o cerrada.

- **Tópicos suscritos:**

- `/hardware/base/scan` – Barrido láser 2D de la base.

- **Servicios ofrecidos:**

- `/hardware/door_detector/check` – Regresa `success=true` si la puerta está abierta (distancia promedio central mayor a 0.75 m), `false` si está cerrada.

- `scanner_node` (nombre interno del nodo: `head_control_node`)

Puente entre un punto 3D de interés y el objetivo de cabeza: al recibir un punto en `/hardware/head/look_at`, calcula pan/tilt vía TF respecto al marco de cámara/cuello, aplica límites articulares y una zona muerta, y publica el objetivo resultante. También reacciona al estado de navegación para evitar mover la cabeza durante ciertas fases.

- **Tópicos publicados:**

- `/hardware/head/goal_pose` – Objetivo de pan/tilt calculado para el *driver* de cabeza.

- **Tópicos suscritos:**

- `/hardware/head/look_at` – Punto 3D de interés (publicado por `Hardware::LookAt`).
 - `/hardware/head/current_pose` – Pose actual de la cabeza.
 - `/navigation/status` – Estado de la meta de navegación en curso.

- `simple_justina_sim` (sólo en configuración de simulación, `justina_sim.launch.xml`)

Sustituye a `hardware_tools` y a los *drivers* reales cuando no hay robot físico: simula la base, las articulaciones y el láser, e implementa directamente los servicios de movimiento que `Hardware.hpp` espera de cabeza/torso/brazos.

- **Tópicos publicados:**

- `/hardware/base/odom`, `/hardware/joint_states`, `/hardware/base/scan` – Odometría, articulaciones y láser simulados.
 - `/hardware/head/current_pose`, `/hardware/torso/current_pose`, `/hardware/arms/left_arm/current_pose`, `/hardware/arms/right_arm/current_pose` – Poses actuales simuladas.
 - `/navigation/initialpose` – Reenvía la pose inicial dada por RViz.
- **Tópicos suscritos:**
 - `/hardware/base/cmd_vel` – Velocidad comandada de la base.
 - `/hardware/head/goal_pose`, `/hardware/torso/goal_pose`, `/hardware/arms/left_arm/goal_pose`, `/hardware/arms/right_arm/goal_pose` – Objetivos de pose simulados.
 - `/initialpose`, `/navigation/initialpose` – Pose inicial desde RViz.
- **Servicios ofrecidos:**
 - `/hardware/head/move_head`, `/hardware/torso/move_torso`, `/hardware/arms/move_left_arm`, `/hardware/arms/move_right_arm` – Simulan el movimiento solicitado y actualizan la pose publicada.
- **kinect_tilt_motor_node** (opcional, `use_kinect`)

Traduce el *tilt* actual de la cabeza en un comando USB directo al motor de inclinación físico del Kinect v1.

 - **Tópicos suscritos:**
 - `/hardware/head/current_pose` – Pose actual de la cabeza, de donde extrae el componente de *tilt*.
- **depth_to_pointcloud** (utilidad de depuración, no forma parte del *pipeline* activo)
 - **Tópicos publicados:**
 - Nube de puntos (`PointCloud2`) reconstruida.
 - **Tópicos suscritos:**
 - Imagen de profundidad y `camera_info` correspondiente.
- **head_node** (repositorio `src_Robotino`)

Recibe posiciones objetivo de pan/tilt y las escribe a los servos Dynamixel de la cabeza vía el SDK sobre RS-485.

 - **Tópicos publicados:**
 - `/hardware/head/joint_states` – Articulaciones de la cabeza (remapeado a `/hardware/joint_states` por `justina_hardware`).
 - `/hardware/head/current_pose` – Posiciones actuales medidas de pan/tilt.
 - `/hardware/head/goal_reached` – Indica si la meta fue alcanzada.

- **Tópicos suscritos:**

- `/hardware/head/goal_pose` – Objetivo de pan/tilt (interfaz alterna basada en tópico; en el sistema activo lo publica `scanner_node`).
- `/navigation/navigation_ctrl/stop` – Detiene el movimiento ante una señal de paro.

- **Servicios ofrecidos:**

- `/hardware/head/move_head` – Mueve la cabeza a un arreglo de posiciones pan/tilt (usado por `Hardware::MoveHead`).

- `torso_node` (repositorio `src_Robotino`)

Controla el eje lineal del torso mediante el controlador Jrk, con retroalimentación analógica de posición (rango `feedback_min/ feedback_max`) y confirmación de meta alcanzada tras varias lecturas estables.

- **Tópicos publicados:**

- `/hardware/torso/joint_states` – Articulación del torso (remapeada a `/hardware/joint_states`).
- `/hardware/torso/current_pose` – Posición actual medida.
- `/hardware/torso/goal_reached` – Indica si la meta fue alcanzada.

- **Tópicos suscritos:**

- `/hardware/torso/goal_pose` – Interfaz alterna basada en tópico para comandar posición.

- **Servicios ofrecidos:**

- `/hardware/torso/move_torso` – Mueve el torso a una posición (usado por `Hardware::MoveTorso`).

- `left_arm_node / right_arm_node` (mismo ejecutable `arms_node`, repositorio `src_Robotino`)

Reciben posiciones articulares de un brazo y las escriben a sus servos Dynamixel, aplicando la compensación de gravedad por *offsets* si está habilitada.

- **Tópicos publicados:**

- `/hardware/arms/left_arm/joint_states` y `/hardware/arms/right_arm/joint_states` – Estado articular de cada brazo (remapeado a `/hardware/joint_states`).
- `/hardware/arms/left_arm/current_pose` y `/hardware/arms/right_arm/current_pose` – Posición actual medida (consumida por `Hardware.hpp`).
- `/hardware/arms/left_arm/goal_reached` y `/hardware/arms/right_arm/goal_reached`

- **Tópicos suscritos:**

- `/hardware/arms/left_arm/goal_pose` y `/hardware/arms/right_arm/goal_pose` – Interfaz alterna basada en tópico.

- `/navigation/navigation_ctrl/stop` – Detiene el movimiento ante una señal de paro.

- **Servicios ofrecidos:**

- `/hardware/arms/move_left_arm / /hardware/arms/move_right_arm`
– Mueven el brazo correspondiente a un arreglo de posiciones con velocidad dada (usado por `Hardware::MoveLeftArm/ MoveRightArm` y por `manipulation_controller`).

- `left_gripper_node / right_gripper_node` (mismo ejecutable `grippers_node`, repositorio `src_Robotino`)

Traducen comandos de pinza a tramas del protocolo binario propio (8 bytes) hacia el microcontrolador RP2350 de cada pinza, por TCP/IP.

- **Tópicos publicados:**

- `/hardware/grippers/left_gripper/joint_states` y `/hardware/grippers/right_gripper/joint_states`
– Estado articular de cada pinza (remapeado a `/hardware/joint_states`).

- **Tópicos suscritos:**

- `/navigation/navigation_ctrl/stop` – Detiene el movimiento ante una señal de paro.

- **Servicios ofrecidos:**

- `/hardware/grippers/move_left_gripper / /hardware/grippers/move_right_gripper`
– Ejecuta `open/close/stop/query_status` sobre la pinza correspondiente y regresa estado, corriente y posición (usado por `Hardware::MoveLeftGripper/ MoveRightGripper`).

- `joint_states_parser_node` (repositorio `src_Robotino`)

Acumula las lecturas parciales de articulaciones de todos los *drivers* y publica un `JointState` completo, en un orden fijo, para `MoveIt2`.

- **Tópicos publicados:**

- `/manipulation/joint_states` – Estado articular completo del robot (torso, ambos brazos, ambas pinzas, cabeza y ruedas de la base).

- **Tópicos suscritos:**

- `/hardware/joint_states` – Estado articular parcial agregado (vía los *remaps* de `justina_hardware`) de cada *driver*.

- `base_node` (nombre interno del nodo: el nombre pasado a `RTNode`; repositorio `src_Robotino`)

Driver principal de la base Robotino 4: conecta por TCP al servidor *RobotinoAPI2* del fabricante y expone tracción omnidireccional, *bumper*, lecturas de motores, energía, IMU y E/S digital.

- **Tópicos publicados:**

- `/hardware/base/bumper` – Estado del parachoques.
- `/hardware/base/motor_readings` y `/hardware/base/motor_error_readings` – Lecturas y errores de los 3 motores omnidireccionales.
- `/hardware/base/joint_states` – Estado de las ruedas (remapeado a `/hardware/joint_states`).
- `/hardware/base/battery_status` – Estado de la batería.
- `/hardware/base/imu` – Lectura del giroscopio.
- `/hardware/base/digital_readings` – Entradas digitales (incluye la señal de paro que consume `Stop_node`).
- `/hardware/base/encoder_readings` – Lecturas de encoders.

- **Tópicos suscritos:**

- `/hardware/base/cmd_vel` – Velocidad omnidireccional comandada (publicada por `Hardware::MoveBase`).
- `/hardware/base/set_digital_values` – Escribe salidas digitales.

- **Servicios ofrecidos:**

- `/hardware/base/cmd_vel_enable` – Habilita o deshabilita la tracción.
- `/hardware/base/set_velocity_limits` – Ajusta los límites de velocidad lineal/angular.
- `/hardware/base/set_encoder_position` – Reinicia la posición de un encoder.

- **robotino_odometry_node** (repositorio `src_Robotino`)

Calcula y publica la odometría de la base a partir de las lecturas del driver, por separado del nodo principal.

- **Tópicos publicados:**

- `/hardware/base/odom` – Odometría de la base; también difunde la TF `odom→base_link` si `launch_odom_tf` está activo.

- **Servicios ofrecidos:**

- `/hardware/base/reset_odometry` – Reinicia la odometría acumulada.

- **Stop_node** (nombre interno del nodo: `digital_stop_node`; repositorio `src_Robotino`)

Traduce una entrada digital física (índice 6 del arreglo de entradas digitales, cableada a un botón/interruptor de paro) en la señal de alto que consumen navegación, brazos, cabeza y pinzas.

- **Tópicos publicados:**

- `/navigation/navigation_ctrl/stop` – Señal de paro (invertida respecto a la entrada digital).

- **Tópicos suscritos:**

- `/hardware/base/digital_readings` – Entradas digitales de la base.

- `urg_node` (paquete de terceros, repositorio `src_Robotino`)

Driver de terceros (paquete ROS `urg_node`, fabricante Hokuyo) para el LiDAR 2D de la base. Se configura con puerto serial `/dev/lidar` y un campo de visión acotado a ± 90 . No se documenta su implementación interna por ser un paquete de terceros; se integra bajo `/hardware/base/scan` (remapeado desde su tópico de salida por defecto), consumido por `door_detector`, `leg_finder_node` y `Navigation`.

A.0.4. Knowledge

El módulo *Knowledge* se encarga de mantener y exponer el estado del mundo del robot en tiempo de ejecución (personas, objetos, ubicaciones conocidas, atributos del robot y analítica de percepción), así como de proveer un banco de frases parametrizables que el módulo HRI utiliza para generar diálogo, resolviendo las referencias de esas frases contra el estado del mundo.

Paquetes

- `knowledge_bank`

Paquete de datos (sin nodos) que aloja los dominios estáticos del mundo (`domains/locations.yaml`, `objects.yaml`, `people.yaml`), el banco de frases organizado por contexto (`phrases/gpsr.yaml`, `hri.yaml`, `inspection.yaml`, `laundry.yaml`, `pnf.yaml`, `restaurant.yaml`), los diccionarios de vocabulario (`dictionaries/`: bebidas, nombres, objetos, órdenes, etc.) y el estado en tiempo de ejecución (`runtime/world.yaml`) persistido por `world_state_node`.

- `knowledge_manager`

Implementa los nodos que exponen el estado del mundo y el banco de frases como servicios de ROS 2: `world_state_node` y `phrase_manager_node`.

- `knowledge_tools`

Librería cliente (clase `Knowledge`) que encapsula las llamadas a los servicios `/knowledge/knowledge_manager/world/get_state`, `/knowledge/knowledge_manager/world/...` y `/knowledge/knowledge_manager/phrases/get_phrase`, para que otros módulos (Executor, Supervisor, HRI) puedan consultar y actualizar el estado del mundo sin reimplementar la lógica de cliente.

Nodos

■ `world_state_node`

Mantiene en memoria el estado del mundo (robot, ubicaciones conocidas, objetos, personas y analítica de percepción); lo construye a partir de los dominios estáticos de `knowledge_bank` al iniciar y lo persiste en `runtime/world.yaml` después de cada actualización. Atiende las peticiones de lectura y escritura en grupos de callbacks independientes, con un *mutex* de lecturas/escrituras concurrente para permitir múltiples lecturas simultáneas.

● Servicios ofrecidos:

- `/knowledge/knowledge_manager/world/get_state` – Consulta el estado del mundo por categoría, identificador y atributo (opcionales); devuelve el resultado serializado en JSON.
- `/knowledge/knowledge_manager/world/update_state` – Actualiza el atributo de una categoría (robot, objeto, persona o analítica) y persiste el nuevo estado.

■ `phrase_manager_node`

Carga el banco de frases de `knowledge_bank/phrases` agrupado por contexto y clave; ante una petición, selecciona una frase aleatoria para la clave solicitada y resuelve los *placeholders* de la forma `{categoria:atributo:id}` (o el formato heredado `categoria_atributo[_id]`) consultando el estado del mundo, antes de devolver el texto final.

● Servicios ofrecidos:

- `/knowledge/knowledge_manager/phrases/get_phrase` – Devuelve una frase resuelta para una clave y un contexto (opcional) dados.

● Servicios utilizados:

- `/knowledge/knowledge_manager/world/get_state` – Consulta los valores del estado del mundo necesarios para resolver los *placeholders* de la frase seleccionada.

A.0.5. Manipulation

El módulo *Manipulation* se encarga de planear y ejecutar las trayectorias de brazos y torso necesarias para tomar, colocar y manipular objetos. Actúa como puente entre los consumidores de manipulación del robot (por ejemplo, la FSM **Take** del Executor) y MoveIt2, que se usa únicamente como motor de planeación cinemática y de colisiones (IK vía TRAC-IK, planeación con OMPL/RRTConnect), sin controlar el hardware directamente. La ejecución real sobre los servomotores se delega a los nodos de **Hardware** (`arms_node`, controlador de torso).

Paquetes

- **manipulation_planner**

Nodo puente entre los consumidores de manipulación y MoveIt2. Recibe una pose objetivo, valida el grupo de planeación y el *tip link*, lee el estado articular real desde `/hardware/joint_states`, solicita la planeación a MoveIt2 (forzando `pipeline_id=ompl`, `planner_id=RRTConnect`) y devuelve la trayectoria articular resultante. Opcionalmente coordina el filtro de nube de manipulación y la exclusión del objeto objetivo para que MoveIt2 no lo trate como obstáculo al planear el acercamiento.

- **manipulation_controller**

Ejecuta una trayectoria ya planeada: recibe la lista de *waypoints* (bloques de torso + 6 articulaciones del brazo), y por cada bloque envía en paralelo las peticiones correspondientes al brazo (izquierdo o derecho, según el grupo de planeación) y al torso, esperando la confirmación de ambos antes de continuar con el siguiente bloque.

- **manipulation_tools**

Paquete de utilidades comunes de la pila de manipulación. Contiene la librería cliente **Manipulation** (patrón *Facade*), que expone **PlanTrajectory** y **FollowTrajectory** como llamadas síncronas sin exponer el manejo asíncrono de clientes de ROS 2 a quien la usa (p. ej. `executor_tools`); el nodo `manipulation_cloud_filter_node`; y el *launch* principal (`manipulation.launch.xml`) que levanta bajo el *namespace* `/manipulation` tanto `move_group` (MoveIt2) como `manipulation_planner` y `manipulation_controller`, remapeando el estado articular a la fuente canónica `/hardware/joint_states`.

- **move_it**

Paquete de configuración generado con el *MoveIt Setup Assistant* para el modelo cinemático de Justina: URDF/SRDF, grupos de planeación (`left_arm_group`, `right_arm_group`), `kinematics.yaml` (solver `TRAC_IKKinematicsPlugin`), límites articulares y controladores. Lanza el nodo `move_group` de MoveIt2, que al ser un nodo de terceros no se documenta a detalle en esta sección.

- **trac_ik**

Librería de terceros (TRAC Labs Robotics) usada como *plugin* de cinemática inversa por los grupos de planeación de MoveIt2. Se incluye en el árbol de código como dependencia vendorizada; no aporta nodos propios del proyecto.

Nodos

- **manipulation_planner_node**

Traduce una pose objetivo del efector final en una trayectoria articular planeada por MoveIt2, validando el grupo de planeación y el *tip link*, y coordinando opcionalmente el filtro de nube y la limpieza del *octomap* antes de planear.

- **Tópicos suscritos:**

- `/hardware/joint_states` – Estado articular real del robot (fuente canónica), usado como punto de partida de la planeación.

- **Servicios ofrecidos:**

- `/manipulation/manipulation_planner/plan_trajectory` – Recibe una pose objetivo, grupo de planeación y *tip link*, y devuelve los nombres de articulaciones y los *waypoints* de la trayectoria planeada.

- **Servicios utilizados:**

- `/manipulation/plan_kinematic_path` – Solicitud `GetMotionPlan` al `move_group` de MoveIt2 (OMPL, RRTConnect).
- `/manipulation/cloud_filter/enable` – Activa o desactiva el filtro de nube de manipulación justo antes y después de planear.
- `/manipulation/clear_octomap` – Limpia el *octomap* de MoveIt2 antes de solicitar una nueva planeación.

- **`manipulation_controller_node`**

Ejecuta una trayectoria ya planeada, enviando bloque a bloque (torso + brazo) las posiciones articulares a los nodos de hardware correspondientes, y reporta éxito o falla por bloque.

- **Servicios ofrecidos:**

- `/manipulation/manipulation_controller/follow_trajectory` – Recibe el grupo de planeación, los nombres de articulaciones y los *waypoints* de una trayectoria, y la ejecuta bloque por bloque.

- **Servicios utilizados:**

- `/hardware/arms/move_right_arm` – Mueve el brazo derecho al bloque de posiciones articulares correspondiente.
- `/hardware/arms/move_left_arm` – Mueve el brazo izquierdo al bloque de posiciones articulares correspondiente.
- `/hardware/torso/move_torso` – Mueve el torso a la posición del bloque actual.

- **`manipulation_cloud_filter_node`**

Filtra la nube de puntos cruda de la cámara a una región de interés (ROI) acotada al volumen útil de los brazos, la reduce por *voxelización* y la publica para alimentar el *octomap* de MoveIt2, evitando que el suelo o puntos fuera de alcance contaminen la planeación. Arranca apagado y sólo se activa bajo demanda (p. ej. desde `manipulation_planner_node`) para no procesar nube de forma innecesaria.

- **Tópicos publicados:**

- `/manipulation/camera/points_filtered` – Nube de puntos filtrada y recortada al ROI de manipulación.
- `/manipulation/cloud_filter/debug_markers` – Marcadores de depuración del ROI y la voxelización (opcional).

- **Tópicos suscritos:**

- `/hardware/camera/points` – Nube de puntos cruda de la cámara del robot.

- **Servicios ofrecidos:**

- `/manipulation/cloud_filter/enable` – Activa o desactiva el procesamiento del filtro bajo demanda.

A.0.6. Navigation

El módulo *Navigation* se encarga de localizar al robot, mantener un mapa de obstáculos dinámicos superpuesto al mapa estático, planear rutas sobre ese mapa, y ejecutarlas con evitación reactiva de obstáculos. Es el módulo con más paquetes del repositorio (14), aunque no todos forman parte del sistema activo de Justina: varios existen específicamente para el backend alternativo del robot Takeshi (HSR) o para la simulación en Webots, y se documentan como tales.

Arquitectura en capas (sistema activo): `nav_controller_node` y `locations_manager_node` son la interfaz pública (usada por `navigation_tools` y por el resto del sistema) y corren en el *namespace* raíz, no bajo `/navigation`. Por debajo, `mvn_pln` (`motion_planner_node`) orquesta una maniobra completa hacia una meta, apoyándose en `path_planner` (ruta A* sobre el mapa aumentado), `simple_move` (seguimiento de trayectoria/control de velocidad) y `potential_fields` (evitación reactiva superpuesta al `cmd_vel`). El mapa aumentado lo mantienen `map_augmenter` y `obstacle_detector` (obstáculos dinámicos por LiDAR/nube) a partir del mapa estático servido por `nav2_map_server` y localizado por `nav2_amcl` (modo AMCL) o `slam_toolbox` (modo SLAM/mapless, mismo resto de la pila).

Paquetes

- `navigation_tools`

Fachada síncrona (clase `Navigation`) sobre ir a una ubicación conocida y registrar la pose actual como ubicación nueva. Aloja también un *launch* propio (no es el usado por `justina.launch.xml`, que en su lugar incluye directamente `navigation_start`).

- `navigation_controller` (dos paquetes: `locations_manager`, `nav_controller`)

La interfaz pública del módulo, corriendo ambos en el *namespace* raíz. `locations_manager` guarda/consulta ubicaciones nombradas (`config/known_locations.yaml`) y difunde su TF cada 0.5 s para visualización. `nav_controller` traduce ir a ubicación conocida.^{en} una meta de pose concreta, calcula un *timeout* proporcional a la distancia a recorrer, y difunde la señal de paro de navegación que consumen los *drivers* de hardware.

- **navigation_planner** (paquete de agrupación; siete paquetes ROS propios más dos exclusivos del backend Takeshi)
 - **navigation_start** – Paquete de sólo *launch*/configuración/mapas (sin nodos propios): aloja `navigation_justina_amcl.launch.xml` (modo AMCL, el que usa `justina.launch.xml` por defecto), `navigation_justina_slam.launch.xml` (mismo resto de la pila, localización con `slam_toolbox` en vez de AMCL, seleccionado con el argumento `mapless=true` de `justina.launch.xml`), los mapas estáticos y de prohibición por escenario (incluye mapas de RoboCup 2026 y del Torneo Mexicano de Robótica 2026), y variantes para el backend Takeshi/HSR (`navigation_takeshi*.launch.xml`, `slam_navigation.launch.xml`) no usadas por Justina.
 - **mvn_pln** – Orquesta una maniobra completa hacia una meta: pide ruta a `path_planner`, la ejecuta con `simple_move`, coordina `potential_fields`, y reacciona a paros y a la señal de riesgo de colisión.
 - **path_planner** – Búsqueda de ruta A* (con opción de movimientos diagonales) sobre el mapa estático o el aumentado.
 - **simple_move** – Sigue una ruta/objetivo con control proporcional de velocidad lineal/angular; puede orientar la cabeza en la dirección de movimiento.
 - **potential_fields** – Campo potencial reactivo (repulsión por LiDAR y nube de puntos) superpuesto al `cmd_vel` para evitar colisiones no vistas por el mapa; incluye `publish_enable.py`, un script auxiliar.
 - **map_augmenter** – `map_augmenter_node` combina el mapa estático, el de prohibición y los obstáculos dinámicos en un "mapa aumentado" (y su versión de costo) para planeación; `obstacle_detector_node` detecta esos obstáculos dinámicos a partir del LiDAR y la nube de la cámara. El paquete también compila `cloud_filter_node`, que está comentado en el *launch* activo (no corre).
 - **augment_gridmap_online** – Herramienta manual: deja que un operador añada un obstáculo permanente al mapa de prohibición haciendo clic en un punto en RViz.
 - **hardware_controller** (no usado por Justina) – `head_controller` y `arm_controller` traducen entre la convención propia de Justina (`goal_pose/current_pose`) y la interfaz estándar de ROS 2 `JointTrajectoryController` (`trajectory_msgs/JointTraj`).

`control_msgs/JointTrajectoryControllerState`). Sólo se referencia desde `slam_navigation.launch.xml` y `navigation_namespace.launch.xml`, ambos orientados al backend Takeshi/HSR – no alcanzables desde `justina.launch.xml`.

- `map_server_bypass` (**no usado por Justina**) – Puentea el mapa de Google Cartographer (`/cartographer_map`) hacia la interfaz `GetMap` estándar que esperan `path_planner/map_augmenter`. Sólo se usa desde `slam_navigation.launch.xml` (Takeshi/HSR con Cartographer), no desde el *launch* de Justina.

- `mapless_navigation` (dos paquetes: `mapless_nav`, `debugger`; **ninguno se usa en el robot real**)

`mapless_nav` implementa navegación reactiva sin mapa: campos potenciales, navegación por árbol de comportamiento (`bt_navigation.py`), y un modelo recurrente entrenado (`RNN_mapless.py` + `TorchModels/RNN.pth`) que decide velocidades directamente a partir de la percepción; sólo se lanza desde el entorno de simulación Webots (`justina_mapless_wbt.launch.xml`), no desde `justina.launch.xml`. `debugger` no es un nodo: es una librería cliente (`DebugClient`, con API idéntica en C++ y Python) que cualquier nodo puede usar para enviar mensajes de estado a un tablero unificado en terminal (`debug_manager.py`), ejecutado manualmente por quien depura, no por ningún *launch*.

Nodos

- `nav_controller_node`

Interfaz pública para ir a una ubicación": resuelve el nombre contra `locations_manager`, publica la meta, calcula un *timeout* proporcional a la distancia (con mínimo y máximo configurables) y espera el resultado.

- **Tópicos publicados:**

- `/nav_control/goal` – Meta de navegación (consumida por `mvn_pln_node`).
- `/navigation/navigation_ctrl/stop` – Señal de paro que consumen los *drivers* de *Hardware* (brazos, cabeza, pinzas) y `gui_node`.

- **Tópicos suscritos:**

- `/navigation/status` – Estado de la meta en curso (publicado por `mvn_pln_node`).

- **Servicios ofrecidos:**

- `/nav_controller/go_to_location` – Usado por `supervisor_manager_node` para navegación correctiva directa.

- **Servicios utilizados:**

- `/locations_manager/get` – Resuelve el nombre de la ubicación a una pose.

■ **locations_manager_node**

Mantiene el catálogo de ubicaciones nombradas y difunde su TF.

- **Tópicos publicados:** TF de cada ubicación conocida (cada 0.5s, para visualización en RViz).
- **Servicios ofrecidos:**
 - `/locations_manager/add` – Agrega una ubicación con pose explícita (usado por `Navigation::AddLocationPose`).
 - `/locations_manager/get` – Consulta la pose de una ubicación por nombre.
 - `/locations_manager/save` – Guarda la pose actual del robot como una ubicación nueva (usado por `Navigation::AddLocation`, la primitiva `save_loc` del `Executor`).

■ **mvn_pln** (nombre interno del nodo: `motion_planner_node`)

Orquesta una maniobra de navegación completa: pide una ruta, la despacha a `simple_move` y reacciona a obstáculos, paros y resultado final.

- **Tópicos publicados:**
 - `/navigation/status` – Estado de la meta (`GoalStatus`).
 - `/navigation/potential_fields/enable`, `/navigation/potential_fields/enable_c` – Activan los campos potenciales por LiDAR/nube en `potential_fields`.
 - `/simple_move/goal_path`, `/simple_move/goal_dist_angle`, `/simple_move/stop` – Comandan a `simple_move`.
- **Tópicos suscritos:**
 - `/nav_control/goal` – Meta de navegación (nombre absoluto codificado en el nodo; el *remap* a `move_base_simple/goal` presente en el *launch* no aplica sobre un nombre absoluto y no tiene efecto).
 - `/stop`, `/navigation/stop` – Señales de paro general y de navegación.
 - `/simple_move/goal_reached` – Resultado de `simple_move`.
 - `/navigation/set_patience`, `/navigation/potential_fields/collision_risk` – Ajuste de reintentos y riesgo de colisión reportado por `potential_fields`.
- **Servicios utilizados:**
 - `path_planner/plan_path_with_static / _augmented` – Solicita la ruta a `path_planner`.
 - Servicios de `map_augmenter` para obtener el mapa aumentado y su costo, y para consultar si hay obstáculos o si el robot está dentro de uno.

■ **path_planner** (nombre interno del nodo: `path_planner_node`)

Busca una ruta punto a punto con A* sobre el mapa solicitado.

- **Servicios ofrecidos:**

- `path_planner/plan_path_with_static / _augmented` – Ruta sobre el mapa estático o el aumentado con obstáculos dinámicos.

- **Servicios utilizados:**

- Servicios `GetMap` de `map_augmenter` (estático, de costo estático, aumentado, de costo aumentado).

- `simple_move` (nombre interno del nodo: `simple_move_node`)

Convierte una ruta/objetivo en comandos de velocidad, con control proporcional y orientación de cabeza opcional hacia la dirección de movimiento.

- **Tópicos publicados:**

- `/hardware/base/cmd_vel` – Velocidad comandada (*remap* de `/cmd_vel` en el *launch*).
- `/simple_move/goal_reached` – Resultado.
- Objetivo de cabeza (cuando `move_head` está activo).

- **Tópicos suscritos:**

- `/simple_move/goal_path`, `/simple_move/goal_dist_angle`, `/simple_move/stop` – Comandados por `mvn_pln`.
- `/stop`, señal de paro de navegación, y fuerza de rechazo/riesgo de colisión reportados por `potential_fields`.

- `potential_fields`

Calcula un campo de repulsión a partir del LiDAR y/o la nube de puntos y lo superpone al `cmd_vel` de `simple_move` para evitar colisiones con obstáculos no reflejados en el mapa.

- **Tópicos publicados:**

- `/hardware/base/cmd_vel` – Corrige la velocidad comandada (*remap* de `/cmd_vel`).
- `/navigation/potential_fields/collision_risk` – Alerta de riesgo de colisión para `mvn_pln`.

- **Tópicos suscritos:**

- `/navigation/potential_fields/enable / enable_cloud` – Activan la repulsión por LiDAR/nube.
- `/simple_move/goal_path` – Referencia de ruta.
- LiDAR y nube de puntos de la cámara.

- `map_augmenter_node`

Combina el mapa estático, el de prohibición y los obstáculos dinámicos en un mapa aumentado y su versión de costo, para que `path_planner` planee evitándolos.

- **Tópicos publicados:** Mapa aumentado (`OccupancyGrid`) y mapa de planeación.
- **Tópicos suscritos:**
 - `clicked_point` – Punto agregado manualmente desde RViz (vía `map_enhancer`).
 - Mapa de obstáculos dinámicos de `obstacle_detector_node`.
- **Servicios ofrecidos:**
 - Cuatro servicios `GetMap` (estático, de costo estático, aumentado, de costo aumentado) y dos `Trigger` (`are_there_obstacles`, `is_inside_obstacles`).
- **Servicios utilizados:**
 - `GetMap` del servidor de mapa estático y del de prohibición.

■ `obstacle_detector_node`

Detecta obstáculos dinámicos a partir del LiDAR y/o la nube de puntos, filtrando por ventana espacial configurable, y los publica como `OccupancyGrid` con decaimiento en el tiempo.

- **Tópicos publicados:** Mapa de obstáculos dinámicos (`OccupancyGrid`), consumido por `map_augmenter_node`.
- **Tópicos suscritos:** LiDAR y nube(s) de puntos de la cámara.
- **Servicios utilizados:**
 - `GetMap` del servidor de mapa estático (para saber qué celdas ya son obstáculo conocido).

■ `map_enhancer` (nombre interno del nodo: `augment_gridmap_online_node`)

Herramienta manual para agregar obstáculos permanentes al mapa de prohibición desde RViz.

- **Tópicos publicados:** Mapa de prohibición aumentado, sus metadatos, y un marcador de depuración.
- **Tópicos suscritos:**
 - `clicked_point` – Punto seleccionado en RViz.
 - Mapa de prohibición fijo.
- **Servicios ofrecidos:**
 - Limpiar el mapa aumentado (`Empty`) y consultarlo (`GetMap`, remapeado a `prohibition_map` para `map_augmenter_node`).

A.0.7. Planning

El módulo *Planning* traduce entradas en lenguaje natural (o metas ya estructuradas) en un plan lineal de instrucciones para el robot. La entrada llega vía *semantic_parser* (un LLM externo – Qwen – o un analizador basado en spaCy) o directamente como una lista de metas `goal(args)`; ambas rutas terminan generando hechos para un motor de reglas CLIPS (`clips_node`), que produce el plan y lo publica en `/planning/clips/results`, consumido por `supervisor_plan_node` (véase *Supervisor*).

Paquetes

- `clips_node` (paquete ROS `clips_node`, Python)

Envuelve el motor de reglas CLIPS (librería `clips`) en un nodo de ROS 2: carga uno de varios conjuntos de reglas según el parámetro `plan_name` (`goals_planning`, `gpsr_planning`, `hri_planning`, `laundry_planning`, `pnplanning_Justina/_Takeshi`, `restaurant_planning`, `robot_inspection` – correspondientes a categorías de prueba de RoboCup@Home), expone servicios para insertar hechos y disparar la planeación, y publica el plan resultante. También aloja dos nodos adicionales: `goals_to_clips_node` (activo) convierte una lista JSON de metas `goal(args)` – típicamente producidas por un LLM – en hechos CLIPS, los inserta en lote y dispara la planeación; y `natural_to_clips.py` (registrado como ejecutable instalable, pero **no referenciado por ningún *launch* del repositorio**) traduciría oraciones libres a hechos CLIPS a través de un servicio curiosamente alojado bajo `/hri/listener/natural_to_clips/interpret` en vez de bajo `/planning`.
- `clips_launch`

Paquete de sólo *launch*: `clips.launch.xml` arranca siempre `clips_node`; arranca `goals_to_clips_node` si `use_goals_to_clips` está activo (lo está por defecto, sobreescrito a `true` desde `planning_tools`); y deja `primitive_to_clips_node` comentado – no se lanza en la configuración activa.
- `plan_manager_node` (código huérfano, no referenciado por ningún *launch*)

Implementación que se solapa casi por completo con `supervisor_plan_node` del módulo *Supervisor*: se suscribe a `/clips/results` (nótese, sin el prefijo `/planning` que usa la ruta activa), ofrece `/request_instruction` y publica `/instruction_msg`. Todo indica que es una versión anterior de esa lógica que quedó en el árbol de código sin conectarse a ningún *launch*.
- `primitive_to_clips`

Convierte texto de una primitiva (acción del Executor) en un hecho CLIPS, consultando el estado del mundo de *Knowledge* (`world_state_client.hpp`) para dar contexto, y puede insertarlo y disparar la planeación. Su nodo, `primitive_to_clips_node`,

ofrece exactamente el servicio que `planning_tools` espera (`/planning/primitive_to_clips/convert`) pero está comentado en `clips.launch.xml` y por lo tanto **no corre en la configuración activa**.

■ `planning_tools`

Fachada síncrona (clase `Planning`) sobre todos los servicios del *pipeline* de planeación: análisis semántico (Qwen y spaCy), extracción de entidades, traducción de primitivas/oraciones a CLIPS, conversión de metas a CLIPS, solicitud de instrucciones y disparo del motor CLIPS. Aloja el *launch* raíz del módulo (`planning.launch.xml`), que bajo el *namespace* `/planning` conecta `clips_launch` y `semantic_parser`. También incluye `qwen_to_plan.py`, una utilidad de línea de comandos para probar manualmente el *pipeline* `Qwen→metas→CLIPS` (no forma parte de ningún *launch*).

Dos métodos de la fachada no tienen backend activo: `interpretToClips()` apunta a `/hri/listener/natural_to_clips/interpret`, cuya única implementación (`natural_to_clips.py`) no se lanza en ningún *launch*; y `requestInstruction()` apunta a `/planning/request_instruction`, pero el único servicio realmente ofrecido con ese propósito es el `/request_instruction` (sin el prefijo `/planning`) de `supervisor_plan_node` – por lo que, tal como está el código hoy, esa llamada tampoco encontraría servidor.

■ `semantic_parser`

Tres nodos de comprensión de lenguaje natural para comandos estilo GPSR: `qwen_semantic_node` (llama a un servidor HTTP externo de Qwen para convertir una oración en metas `goal(args)` estructuradas; también ofrece un servicio combinado “escuchar + interpretar” que llama directamente al Listener de HRI), `semantic_node` (alternativa basada en spaCy que produce un análisis tipo *Conceptual Dependency*), y `entity_extractor_node` (extrae una entidad nombrada de una categoría dada a partir de una oración, usando los diccionarios de vocabulario del `knowledge_bank` de *Knowledge* directamente por su directorio de instalación, no por servicio ROS).

Anomalía en el launch activo: `semantic_parser.launch.xml` lanza `semantic_node` dos veces bajo el mismo nombre de nodo (`semantic_node`): una vez condicionada a `use_spacy_parser` con servicio `semantic_parser_spacy`, y otra vez incondicionalmente con el nombre de servicio por defecto del código, `semantic_parser` – que es exactamente el mismo nombre de servicio que `qwen_semantic_node` recibe explícitamente por parámetro en ese mismo *launch*. El resultado son dos nodos con el mismo nombre (`semantic_node` duplicado) y dos servidores distintos anunciando el mismo servicio (`/planning/semantic_parser/semantic_parser`, uno respaldado por Qwen y otro por spaCy); sólo leyendo el código no se puede determinar cuál de los dos atiende una llamada dada de `Planning::parseSemanticCommand()`.

Nodos

■ `clips_node`

Motor de planeación: carga el conjunto de reglas CLIPS correspondiente a `plan_name`, atiende peticiones para insertar hechos y disparar la planeación (en un hilo de trabajo separado para no bloquear el *executor* de ROS 2), y publica el plan resultante.

● Tópicos publicados:

- `/planning/clips/results` – Plan generado, como texto serializado.

● Servicios ofrecidos:

- `/planning/clips/start_planning` – Dispara el motor de reglas sobre los hechos ya insertados.
- `/planning/clips/assert_fact` – Inserta un hecho CLIPS individual.
- `/planning/clips/assert_facts_batch` – Inserta varios hechos CLIPS de una vez.

■ `goals_to_clips_node` (nombre interno del nodo: `goals_to_clips`)

Traduce una lista JSON de metas `goal(args)` en hechos CLIPS, los inserta en lote y dispara la planeación.

● Servicios ofrecidos:

- `/planning/goals_to_clips/convert` – Recibe el JSON de metas y regresa el resultado de insertarlas y disparar la planeación.

● Servicios utilizados:

- `/planning/clips/assert_facts_batch` y `/planning/clips/start_planning` – Insertan los hechos generados y disparan el motor CLIPS.

■ `qwen_semantic_node`

Convierte una oración en metas estructuradas llamando a un servidor HTTP externo de Qwen; también ofrece un modo `.escuchar y luego interpretar` para comandos GPSR.

● Servicios ofrecidos:

- `/planning/semantic_parser/semantic_parser` – (nombre de servicio configurado explícitamente por *launch*, ver anomalía arriba) Interpreta una oración ya transcrita y regresa metas estructuradas.
- `/planning/semantic_parser/listen_gpsr` – Escucha un comando de voz (vía el Listener de HRI) y lo interpreta con Qwen en una sola llamada.

● Servicios utilizados:

- `/hri/listener/listen` – Captura el comando hablado para el modo `listen_gpsr`.
- `semantic_node` (lanzado dos veces con el mismo nombre; ver anomalía arriba)

Analiza una oración con un *pipeline* de spaCy (modelo `en_core_web_trf`) y produce una representación semántica tipo *Conceptual Dependency*.

 - **Servicios ofrecidos:**
 - `semantic_parser_spacy` (primera instancia, condicionada a `use_spacy_parser`)
 - / `semantic_parser` (segunda instancia, sin condición, nombre por defecto del código – coincide con el servicio de `qwen_semantic_node`) – Ambas bajo `/planning/semantic_parser/`.
- `entity_extractor_node`

Extrae una entidad nombrada de una categoría dada (bebidas, nombres, objetos, etc.) de una oración, usando los diccionarios de `knowledge_bank`.

 - **Servicios ofrecidos:**
 - `/planning/semantic_parser/extract_entity` – Regresa la(s) entidad(es) de la categoría solicitada encontradas en la oración.
- `primitive_to_clips_node` (**no se lanza en la configuración activa** – comentado en `clips.launch.xml`)
 - **Servicios ofrecidos:**
 - `/planning/primitive_to_clips/convert` – Convierte el texto de una primitiva en un hecho CLIPS (consultando el estado del mundo para dar contexto) y puede insertarlo y disparar la planeación.
 - **Servicios utilizados:**
 - `/planning/clips/assert_facts_batch` y `/planning/clips/start_planning`.
- `natural_to_clips` (**no referenciado por ningún *launch***; ejecutable instalado pero huérfano)
 - **Servicios ofrecidos:**
 - `/hri/listener/natural_to_clips/interpret` – Traduciría una oración libre a un hecho CLIPS.
- `plan_manager_node` (código huérfano, **no referenciado por ningún *launch***; ver nota en *Paquetes*)
 - **Tópicos publicados:**
 - `/instruction_msg` – Instrucción despachada.

- **Tópicos suscritos:**
 - `/clips/results` – Plan generado (ruta sin prefijo `/planning`, inconsistente con la activa).
- **Servicios ofrecidos:**
 - `/request_instruction` – Regresa la siguiente instrucción de la cola.

A.0.8. Supervisor

El módulo *Supervisor* cierra el ciclo entre *Planning* y *Executor*: convierte el plan serializado que produce CLIPS en una cola de instrucciones que el Executor va solicitando una a una, valida y clasifica el resultado de cada tarea que el Executor reporta, y (cuando está habilitado) orquesta comportamiento correctivo – reintentos, rebobinar un paso, inyectar una instrucción de emergencia – entre ambos.

Hallazgo importante: en la configuración de *launch* activa (`supervisor_manager/launch/supervisor`) el nodo `supervisor_manager_node` está comentado y por lo tanto **no se lanza**. Es, con mucho, el componente más grande del módulo (más de 4400 líneas), e implementa el coordinador de más alto nivel: la política de recuperación PNP, la inyección de instrucciones correctivas, y la señal `/supervisor/supervisor_manager/allow_next_instruction` que `supervisor_execution_node` necesita para volver a pulsar `/input_trigger` después de cada tarea. Con `supervisor_manager_node` apagado, nada más en el repositorio publica esa señal (se verificó que sólo esos dos nodos referencian `/input_trigger`), por lo que el pulso automático de "siguiente instrucción" depende de que se habilite este nodo; esta sección documenta su implementación tal como existe en el código, señalando explícitamente que está desactivado en el *launch* activo.

Paquetes

■ `supervisor_plan`

Recibe el plan serializado publicado por CLIPS (`/planning/clips/results`), lo parsea en una cola interna de instrucciones secuenciales, y expone `/request_instruction` para que `executor_manager` las vaya pidiendo una a una. Si no hay plan cargado, responde con una instrucción de respaldo ("`Justina listen keyword`") para que el robot siga siendo reactivo. Soporta superponer una instrucción correctiva sin perder la cola original (se reanuda el plan tras ejecutarla) y rebobinar un paso.

■ `supervisor_execution`

Valida y clasifica el resultado de cada tarea que `executor_manager` reporta después de que `executor_worker` la ejecuta. Contiene una clase validadora por cada nombre de acción del Executor (`GoTo`, `Say`, `Listen`, `Check`, `Save`, `Describe`, `Point`, `Find`, `Focus`, `Follow`, `Unfocus`, `MemorizeFace`, `SaveLoc`, `Align`, `Take`, `VerifyTake`, `Drop`, `Analyze`, `Approach`, `Fold`); cada una recibe el argumento original de la acción y el texto crudo devuelto por `executor_tools` (p. ej. `.^rrived_at:`

`cocina: ...")` y lo normaliza a un `TaskExecutionReport` estructurado (`approved`, `status`, `failure_reason` categorizado – argumento inválido, resultado mal formado, *timeout*, falla de navegación, etc.).

- `supervisor_manager` (no se lanza en la configuración activa – ver hallazgo arriba)

Coordinador de más alto nivel: escucha `/system/status`, el resultado validado de `supervisor_execution`, y el estado del plan de `supervisor_plan`; puede pedir rebobinar un paso, habilitar la siguiente instrucción, habilitar la entrega de una instrucción del plan, mandar un comando auxiliar (reintento) o inyectar una instrucción correctiva (p. ej. pedirle a Justina que repita un dato que no entendió). Mantiene clientes propios y directos a Talker, Listener, navegación por ubicaciones conocidas y cabeza, para comportamiento correctivo hablado/de orientación independiente del Executor. Incluye `PnpRecoveryManager` (`pnp_recovery.hpp/cpp`): una política de tiempo de ejecución sobre el plan PNP (*Pick aNd Place*) fijo que emite `pnp_planning_Justina.clp` – inserta `verify_take` después de cada toma exitosa, pospone un objeto fallido mientras el plan nominal continúa, reintenta localmente en el origen antes de transferir al destino, salta los pasos de destino/soltar para objetos nunca verificados como cargados, y usa el tiempo restante en un segundo intento sobre los objetos fallidos.

- `supervisor_system`

Publica `/system/status` cada segundo. **Implementación actual:** el nodo (`supervisor_system_r`, 65 líneas) no inspecciona el grafo de ROS 2 ni verifica la presencia de nodos concretos – publica un mensaje con todos los módulos (`navigation`, `manipulation`, `vision`, `hri`, `executor`, `planner`, `hardware`) marcados como saludables de forma fija (*hardcoded*). La verificación real de nodos activos, terminación y relanzamiento automático que describe el README del paquete no está implementada en este código; es un *stub*/marcador de posición para ese trabajo futuro.

- `surge_et_ambula`

Paquete de sólo *launch* (sin nodos propios) que centraliza el *bringup* de todo el robot: `justina.launch.xml` (robot real, incluye Supervisor, Executor, Planning, Navigation, Manipulation, HRI, Vision, Knowledge y Hardware, más RViz), `justina_sim.launch.xml` (misma composición pero sustituyendo `hardware_tools` por `simple_justina_sim.py`), `map.launch.xml`, y los *launches* alternos para el backend de hardware del robot Takeshi (HSR).

Nodos

- `supervisor_plan_node`

Mantiene la cola de instrucciones del plan activo y la entrega bajo demanda.

- **Tópicos publicados:**

- `/instruction_msg` – Instrucción actualmente despachada (sólo para observabilidad).
- `/supervisor/supervisor_plan/active` – Indica si aún quedan instrucciones por ejecutar en el plan.
- `/supervisor/supervisor_plan/plan_ready` – Pulso que indica que un nuevo plan de CLIPS ya fue parseado.

- **Tópicos suscritos:**

- `/planning/clips/results` – Plan serializado producido por CLIPS.
- `/supervisor/supervisor_manager/instruction` – Instrucción correcta a superponer sobre el plan.
- `/supervisor/supervisor_manager/instruction_command` – Comando auxiliar (p. ej. reintentar).
- `/supervisor/supervisor_manager/allow_plan_instruction` – Habilita la entrega de la siguiente instrucción del plan.
- `/supervisor/supervisor_plan/rewind_one` – Solicita repetir la instrucción anterior.

- **Servicios ofrecidos:**

- `/request_instruction` – Regresa la siguiente instrucción de la cola (o la instrucción de respaldo si no hay plan cargado).

- **supervisor_execution_node**

Autoriza/clasifica el resultado de cada tarea reportada por el Executor, despachando al validador de la clase correspondiente a la acción, y mantiene viva la señal que permite pedir la siguiente instrucción.

- **Tópicos publicados:**

- `/input_trigger` – Pulso que dispara el ciclo de solicitud/ejecución en `executor_manager_node` (emitido en respuesta a `allow_next_instruction`; reintenta si aún no hay suscriptor).
- `/supervisor/supervisor_execution/result` – Reporte estructurado (`TaskExecutionReport`) de la tarea recién validada.

- **Tópicos suscritos:**

- `/supervisor/supervisor_manager/allow_next_instruction` – Señal para volver a pulsar `/input_trigger` (publicada únicamente por `supervisor_manager_node` actualmente deshabilitado).

- **Servicios ofrecidos:**

- `/supervisor/supervisor_executor/authorize` – Recibe la acción, argumento y resultado crudo del Executor; regresa si fue aprobada y el motivo de falla si no.

- **supervisor_system_node** (nombre interno del nodo: `system_status_publisher`)

Publica el estado de salud del sistema una vez por segundo. *Implementación actual: valores fijos, ver nota en Paquetes.*

- **Tópicos publicados:**

- `/system/status` – Estado (fijo) de los módulos de navegación, manipulación, visión, HRI, executor, planner y hardware.

- **supervisor_manager_node** (**no se lanza en la configuración activa**)

Coordinador de más alto nivel y política de recuperación PNP, descrito en *Paquetes*. Interfaz tal como está implementada en el código (no en ejecución):

- **Tópicos publicados:**

- `/supervisor/supervisor_plan/rewind_one`, `/supervisor/supervisor_manager/allow_plan_instruction`, `/supervisor/supervisor_manager/instruction` – Señales de control hacia `supervisor_plan_node` y `supervisor_execution_node` descritas arriba.

- **Tópicos suscritos:**

- `/system/status` – Salud del sistema.
- `/supervisor/supervisor_execution/result` – Resultado validado de la última tarea.
- `/supervisor/supervisor_plan/active` y `/supervisor/supervisor_plan/plan_ready` – Estado del plan.
- `/instruction_msg` – Instrucción en curso (sólo observabilidad).

- **Servicios utilizados:**

- `/hri/talker/talk`, `/hri/listener/listen` – Habla/escucha correctiva independiente del Executor.
- `/nav_controller/go_to_location` – Navegación correctiva a una ubicación conocida.
- `/hardware/head/move_head` – Movimiento correctivo de cabeza.

A.0.9. Vision

El módulo *Vision* se encarga de la percepción visual del robot: detección de objetos 2D/3D (YOLO y OWLv2), analítica sobre esas detecciones (conteo, tamaño, color, comparación), reconocimiento e identificación facial, detección de asientos vacíos, alineación con mesas, y localización de espacios de entrega (mesa/repisa/contenedor) para las tareas de manipulación. Expone todo esto a través de la fachada `vision_tools`. Es, con *Navigation*, uno de los módulos con más nodos activos simultáneamente (14, todos bajo el *namespace* `/vision`).

Paquetes

- **vision_tools**

Fachada síncrona (clase `Vision`) sobre todos los servicios de detección, analítica y reconocimiento facial del módulo. Aloja el *launch* raíz (`vision.launch.xml`, con una cantidad considerable de parámetros afinados a mano, sobre todo para los nodos de `vacancy_detector`) y tres nodos propios: `detect_seats_node`, `Detect_Fusion_Node` e `image_fixed_node` (descritos en *Nodos*).

Un método de la fachada no tiene backend activo bajo el nombre esperado: `Vision::DetectByClass` llama a `/vision/detect_by_class`, pero `Detect_Fusion_Node` – que implementa exactamente esa lógica – registra el servicio como `/detect_by_class`, sin el prefijo `/vision`; hay una línea comentada en el código que sugiere que el nombre con prefijo se consideró en algún momento. Mientras no coincidan, esa llamada de la fachada no encuentra servidor.

- **object_recognizer** (tres paquetes ROS: `owlv2_detect`, `pc_bbox_pose`, `yolo_detect`)

`yolo_detect` corre YOLO11 sobre la imagen de la cámara, publicando la lista de detecciones y ofreciendo detección bajo demanda y el interruptor de detección continua. `owlv2_detect` corre el modelo OWLv2 (detección por texto/*open-vocabulary*) sobre la misma imagen; en el sistema activo se lanzan `owlv2_node` (servicio de detección) y `Analyze_node` (analítica de conteo, tamaño, color y comparación sobre las detecciones de OWLv2, no de YOLO). El paquete también instala `Analyze_Human_node` y `owl_2d`, ninguno de los dos referenciado por `vision.launch.xml`; en particular, `Analyze_Human_node` sería el servidor de `/vision/analyze_person` que consume `Vision::AnalyzePerson()`, por lo que ese método de la fachada tampoco tiene backend activo. `pc_bbox_pose` convierte un *bounding box* 2D más la nube de puntos en una pose 3D.

- **face_recognizer** (paquete ROS `face_recog`)

Reconocimiento facial. El *launch* activo usa `faceFlorence` (`face_recogFlorence2.py`, basado en el modelo Florence-2), que ofrece detectar, identificar, describir y registrar personas por rostro. Los ejecutables `face` (`face_recog.py`) y `facev2` (`face_recogv2.py`) se instalan pero no se lanzan – versiones anteriores del mismo nodo.

- **table_alignment**

Segmenta la nube de puntos para encontrar el plano de una mesa (redonda o rectangular) y calcula el ángulo de alineación entre el robot y su borde.

- **vacancy_detector**

Localiza puntos de entrega libres en tres tipos de superficie – `table_drop_tf_node` (mesa), `shelf_drop_tf_node` (repisa/estante, detectando planos horizontales candidatos) y `container_drop_tf_node` (contenedor/cubeta, apoyándose opcional-

mente en un *bounding box* de OWLv2 para acotar la región de búsqueda antes de procesar geometría) – y publica el resultado como TF. Los tres reutilizan la misma nube de puntos filtrada por `vacancy_cloud_filter_node`: cada uno activa el filtro compartido bajo demanda para no procesar nube constantemente, y todos arrancan desactivados (`active_on_start=false`). Las ubicaciones de referencia de cada superficie viven en archivos de texto planos dentro del propio paquete (`table_drop_points.txt`, `shelf_drop_points.txt`, `container_drop_points.txt`).

Nodos

■ `yolo_detect_node`

Corre YOLO11 sobre la imagen de la cámara y expone las detecciones.

- **Tópicos publicados:**

- `/vision/detections_list` – Lista de detecciones YOLO (consumida por `person_tracker` de HRI).

- **Tópicos suscritos:**

- Imagen de la cámara (parámetro `image_topic`).
- `/vision/enable_continuous` – Activa/desactiva la detección continua (también como servicio, ver abajo).

- **Servicios ofrecidos:**

- `/vision/detection` – Detección bajo demanda.
- `/vision/enable_continuous` – Activa/desactiva la detección continua (usado por `Vision::EnableContinuous` y por `human_tracking_node` de HRI).

■ `owlv2_detect_node` (nombre interno del nodo: `owl_server`)

Corre OWLv2 (detección por texto) sobre la imagen de la cámara.

- **Tópicos publicados:**

- `/vision/detections` – Detecciones de OWLv2 (consumidas por `Analyze_node`).
- `/vision/owl/debug_image` – Imagen de depuración (consumida por `gui_node` de HRI/Hardware).

- **Tópicos suscritos:**

- Imagen de la cámara (parámetro `image_topic`).

- **Servicios ofrecidos:**

- `/vision/owl/owlv2_service` – Detección por texto/consulta abierta (usado por `Vision::DetectOwlV2`, `Detect_Fusion_Node` y `container_drop_tf_node`).

- **Servicios utilizados:**

- `/vision/bbox_to_pose` – Convierte el *bounding box* detectado en una pose 3D.

■ **Analyze_node** (nombre interno del nodo: `vision_analytic_service`)

Analítica sobre las detecciones de OWLv2: conteo (con acumulado histórico e IoU para no contar el mismo objeto dos veces), tamaño, color y comparación entre clases.

- **Tópicos suscritos:**

- `/vision/detections` – Detecciones de OWLv2 que alimentan el historial de conteo.

- **Servicios ofrecidos:**

- `/vision/count_objects`, `/vision/get_by_size`, `/vision/get_by_color`, `/vision/compare_count` – Las cuatro modalidades de analítica (usadas por `Vision::AnalyzeCount/Size/Color/Compare`).

- **Servicios utilizados:**

- `/vision/owl/owlv2_service` – Redetecta bajo demanda cuando el historial no basta.

■ **pc_bbox_pose_node**

Convierte un *bounding box* 2D más la nube de puntos en una pose 3D del objeto detectado.

- **Tópicos suscritos:**

- Nube de puntos de la cámara (parámetro `pointcloud_topic`).

- **Servicios ofrecidos:**

- `/vision/bbox_to_pose` – Usado por `Vision::DetectByClass/GetOwlPose`, `owlv2_detect_node` y `detect_seats_node`.

■ **face_recog_node** (ejecutable `faceFlorence`; nombre interno del nodo: `face_service_node`)

Detecta, identifica, describe y registra personas por reconocimiento facial (modelo Florence-2).

- **Tópicos publicados:**

- `/face_recog/debug_image` – Imagen de depuración (consumida por `gui_node`).

- **Tópicos suscritos:**

- Imagen de la cámara y su `camera_info`.

- **Servicios ofrecidos:**

- `/vision/detect_person`, `/vision/identify_person`, `/vision/describe_person`, `/vision/register_person` – Las cuatro operaciones del pipeline de rostro (usadas por `Vision::DetectPerson/IdentifyPerson/DescribePerson/RegisterPerson`).

■ `detect_seats_node`

Combina detección YOLO y conversión a pose 3D para determinar si hay un asiento libre frente al robot.

● Tópicos publicados:

- `/vision/detect_seats/debug_image` – Imagen de depuración (consumida por `gui_node`).

● Tópicos suscritos:

- Imagen de la cámara (parámetro `image_topic`, sólo para depuración).

● Servicios ofrecidos:

- `/vision/find_seat` – Usado por `Vision::FindSeat`.

● Servicios utilizados:

- `/vision/detection` y `/vision/bbox_to_pose` – Detecta y ubica candidatos a asiento.

■ `Detect_Fusion_Node` (nombre interno del nodo: `detect_fusion_node`)

Elige un detector 2D (YOLO u OWLv2, según el modelo solicitado), obtiene el *bounding box* de la clase pedida, y lo convierte en pose 3D vía `pc_bbox_pose`.

● Servicios ofrecidos:

- `/detect_by_class` – Ver anomalía de nombre en *Paquetes*: la fachada llama a `/vision/detect_by_class`, sin coincidir.

● Servicios utilizados:

- `/vision/detection`, `/vision/owl/owlv2_service` – Los dos detectores disponibles.
- `/vision/bbox_to_pose` – Conversión a pose 3D.

■ `table_segmenter_node`

Segmenta el plano de una mesa en la nube de puntos y calcula el ángulo de alineación del robot respecto a su borde.

● Tópicos publicados:

- Nube de puntos segmentada, ángulo de alineación (`Float64`) y marcadores de depuración (`RViz`).

● Tópicos suscritos:

- Nube de puntos de la cámara (parámetro `pointcloud_topic`).

- **Servicios ofrecidos:**

- `/vision/get_table_alignment` – Usado por `Vision::GetTableAlignment`.

- **`vacancy_cloud_filter`** (nombre interno del nodo: `vacancy_cloud_filter_node`)

Filtro de nube compartido por los tres nodos de `vacancy_detector`: recorta la nube al ROI configurado y la publica sólo mientras está activo; al activarse, pausa opcionalmente la navegación por campos potenciales y el filtro de obstáculos de nube de *Navigation* para no interferir con la detección de espacio.

- **Tópicos publicados:**

- `/vision/vacancy/points_filtered` – Nube filtrada al ROI (parámetro `output_topic`).
- Habilitación de campos potenciales de navegación (normal y de nube), restaurada al desactivarse.

- **Tópicos suscritos:**

- Nube de puntos de la cámara (parámetro `input_topic`).

- **Servicios ofrecidos:**

- `/vision/vacancy_cloud_filter/enable` – Activa o desactiva el filtrado (llamado por los tres nodos `*_drop_tf_node` bajo demanda).

- **Servicios utilizados:**

- Servicio de habilitación del filtro de obstáculos de nube de *Navigation* (parámetro `nav_cloud_filter_enable_service`).

- **`table_drop_tf_node` / `shelf_drop_tf_node` / `container_drop_tf_node`**

Buscan un punto de entrega libre sobre la nube filtrada compartida y lo publican como TF (`table_frame`, `shelf_frame` y `container_frame` respectivamente, con su punto de aproximación `*_drop_start`); `shelf_drop_tf_node` además detecta los planos horizontales candidatos (repisas) antes de elegir sección; y `container_drop_tf_node` puede usar un *bounding box* de OWLv2 como región de búsqueda antes de procesar la geometría, con reintentos si la ventana visual recorta demasiado.

- **Tópicos publicados:**

- Marcadores de depuración (RViz); adicionalmente, la nube recortada de depuración en `container_drop_tf_node`.

- **Tópicos suscritos:**

- Nube filtrada de `vacancy_cloud_filter` (parámetro `pointcloud_topic`).

- **Servicios ofrecidos:**

- `enable` (`SetBool`, uno por nodo) – Activa/desactiva la búsqueda de ese tipo de superficie.

- **Servicios utilizados:**

- `/vision/vacancy_cloud_filter/enable` – Activa el filtro compartido mientras busca.
- `/vision/owl/owlv2_service` – Sólo `container_drop_tf_node`, para acotar la búsqueda por visión antes de procesar geometría.

- `image_fixed_node` (nombre interno del nodo: `hsr_image_enhancer`)

Aplica balance de blancos y realce a la imagen cruda de la cámara del HSR/Takeshi y la republica bajo el tópico canónico de cámara de Justina. Se lanza sin condición en `vision.launch.xml`; en el robot real (base Robotino, cámara vía `openni2_camera`) nada publica en el tópico HSR del que depende, por lo que el nodo simplemente no recibe imágenes y no tiene efecto – es relevante sólo al operar sobre el backend de hardware alterno de Takeshi.

- **Tópicos publicados:**

- `/hardware/camera/image_raw` – Imagen realzada, en el tópico que consume el resto del sistema.

- **Tópicos suscritos:**

- `/head_rgbd_sensor/rgb/image_rect_color` – Tópico de cámara estándar del HSR/Takeshi.

Apéndice B

Protocolo de investigación

Entrevista a usuario final

1. ¿En qué tareas del hogar le gustaría que un robot lo asista?
2. ¿Hay actividades que preferiría hacer usted mismo en lugar de delegarlas a un robot? ¿Por qué?
3. ¿Qué tanto espera que el robot trabaje de forma autónoma en lugar de recibir órdenes explícitas?
4. ¿Cuánto tiempo cree que debería tomarle a un robot realizar sus tareas respecto al tiempo que le tomaría a una persona?
5. Si usted saliera de viaje y el robot se quedara solo en su casa, ¿qué le pediría que hiciera en su ausencia?
6. ¿De qué manera le gustaría poder dar órdenes al robot (escritas, habladas, desde un control, una computadora)?
7. ¿De qué forma se imagina que sería más fácil confirmar que el robot realmente entendió la orden que está por ejecutar?
8. ¿Le gustaría que el robot le reportara el estatus de las actividades encargadas? ¿En qué forma le gustaría que lo hiciera?
9. ¿Qué tan importante le parece que el robot parezca una persona?
10. ¿Cree que el robot debería cambiar su comportamiento dependiendo de con quién interactúa? En caso afirmativo, ¿de qué forma?
11. ¿Qué artículo del hogar le daría más miedo permitir que un robot manipule?
12. ¿Cuáles son los objetos más pesados o frágiles sería útil para usted que el robot pudiera manipular?

13. Además de objetos y bolsas, ¿qué otro tipo de cosas crees que sería útil que el robot pudiera manipular? Ej. Puertas, llaves, cepillos
14. ¿Le gustaría que el robot incluyera un manual para que usted mismo le pudiera dar mantenimiento y hacer reparaciones básicas o preferiría llevarlo con el proveedor directamente ante cualquier falla?
15. ¿Utilizaría el robot únicamente en su hogar o consideraría llevarlo consigo para que lo asista en viajes?
16. ¿La apariencia de qué robot le inspira mayor confianza? ¿Por qué?



(a) Opción 1



(b) Opción 2



(c) Opción 3

17. ¿Cuáles son las principales preocupaciones que le generaría tener un robot de servicio en casa?
18. ¿Qué elementos podría incluir el robot para hacerlo sentir más seguro?

Entrevista al desarrollador de software (General)

1. Cuéntanos acerca de tu experiencia en la RoboCup, enfocada principalmente a la categoría @Home.
2. ¿Cuáles han sido los problemas más importantes que has enfrentado o te has enterado que han enfrentado otros equipos durante una competencia?
3. ¿Cuál consideras que es el hardware mínimo que se requiere para participar en esta categoría? ¿Cuál de esos elementos consideras que tiene un mayor impacto en el desempeño del robot?
4. ¿Consideras que la apariencia del robot afecta la manera en la que el usuario interactúa con él? ¿Crees que este efecto tiene un impacto real en su desempeño?
5. Además de las instrucciones que se le dan a un usuario no experto antes de una prueba, ¿qué otros elementos integrados en el robot has notado que hacen más intuitiva la interacción?

6. ¿Qué otras formas de interacción se han usado cuando el usuario no domina el idioma inglés?
7. ¿Has identificado alguna característica del diseño de los manipuladores, que nosotros u otros equipos hayan implementado, que pudiera incomodar/lastimar al usuario?
8. ¿Qué tareas de manipulación consideras más críticas en la competencia?
9. ¿Cuál consideras una mejor característica al tomar objetos: la velocidad o la precisión? ¿Por qué?
10. ¿Cuál has identificado como el principal motivo por el que a un robot se le caen objetos que ya había tomado?
11. ¿Cuál es el objeto más grande o pesado que se debe tomar en las pruebas?
12. ¿Cuál es el objeto más pequeño y delicado que se ha tenido que manipular en las pruebas?
13. ¿Hay alguna otra característica, además del peso y tamaño, que haga complicado tomar un objeto? (Ej. Rugosidad de la superficie)
14. ¿Qué condiciones de luz has identificado que afectan más a los sistemas de percepción del robot?
15. ¿Has identificado problemas con puntos ciegos del robot en la competencia?
16. ¿Consideras que la medición de par o fuerza en las articulaciones del brazo y gripper sería útil?
17. ¿Qué medidas de redundancia crees que sean más importantes para la seguridad y el rendimiento del robot?
18. ¿Qué elementos de seguridad, además del botón de paro, consideras que es importante incluir en el diseño del robot?

Entrevista al desarrollador de software (Plataforma experimental)

1. ¿Cuáles consideras que son las características clave del manipulador para tomar objetos y cuáles pueden ser compensadas con la posición de la base?
2. ¿Cuál recuerdas como el efector final más efectivo en la competencia?
3. ¿Crees que la integración de una cámara en el gripper mejoraría su habilidad de tomar objetos?

4. ¿Hay alguna característica del robot que consideres importante como el peso o el tamaño? Más allá de las limitaciones que impone el reglamento.
5. En caso de contar con un robot de dos brazos, ¿consideras que deberían ser brazos idénticos o cada brazo debería tener una habilidad diferente? ¿El efector final debería ser intercambiable?
6. En los diseños previos, ¿qué partes del hardware eran más inaccesibles para su mantenimiento?

Entrevista al desarrollador de software (Plataforma comercial)

1. ¿Cuáles consideras que son las mayores virtudes del hardware de Takeshi?
2. Si pudieras modificar algo, ¿lo harías? ¿qué sería?
3. ¿Crees que la integración de la cámara en el gripper mejora su habilidad de tomar objetos o sólo incrementa el tiempo de procesamiento? ¿Sí la utilizan?
4. ¿Considerarías útil que el efector final fuera intercambiable?
5. ¿Qué tareas de manipulación consideras que son las más desafiantes considerando que Takeshi solo cuenta con un brazo?
6. ¿Han tenido que compensar las limitaciones de los grados de libertad que proporciona Takeshi? ¿Cómo lo han hecho?
7. ¿Han tenido que compensar las limitaciones de los grados de libertad que proporciona Takeshi? ¿Cómo lo han hecho?
8. ¿Existe alguna limitación en el mantenimiento del hardware considerando que se trata de un robot comercial?
9. Dado que todos tienen el mismo hardware, ¿qué utilizas como diferenciador?

Riesgos y beneficios anticipados

Riesgos

Esta investigación presenta riesgos mínimos para los participantes. No se utiliza instrumento alguno o contacto con el robot de ningún tipo y únicamente involucra preguntas sobre experiencias técnicas y necesidades cotidianas.

Los riesgos identificados incluyen:

- Posible incomodidad al compartir información sobre necesidades personales de asistencia doméstica.

- Fatiga durante la sesión de entrevista (mitigado mediante sesiones de duración razonable con pausas si es necesario).
- Riesgo mínimo de identificación indirecta a pesar de la codificación de datos.

Beneficios

Los participantes podrán obtener los siguientes beneficios:

- Contribución directa al desarrollo de tecnología de asistencia robótica que puede mejorar la calidad de vida de personas con necesidades similares.
- Oportunidad de reflexionar sobre sus experiencias y necesidades, lo cual puede generar aprendizajes valiosos para su propio contexto.
- Retroalimentación que puede enriquecer su perspectiva sobre el diseño centrado en el usuario.
- Participación en investigación que busca cerrar la brecha entre el desarrollo tecnológico y las necesidades reales de los usuarios finales.

Criterios de selección

Los participantes serán seleccionados mediante muestreo intencional conforme a dos perfiles específicos:

Perfil 1: Equipo de desarrollo

- Ser miembro activo o previo del equipo de desarrollo del robot de servicio doméstico Justina.
- Contar con experiencia de participación en la competencia RoboCup@HOME.
- Tener conocimiento técnico sobre el diseño, programación o implementación de funcionalidades del robot.

Perfil 2: Usuarios potenciales

- Ser personas que requieren asistencia doméstica constante debido a edad avanzada (65 años o más) o alguna condición de discapacidad que limite su autonomía en actividades cotidianas del hogar.
- Residir de forma independiente o semi-independiente en un entorno doméstico.
- Tener capacidad cognitiva para participar en una entrevista y proporcionar consentimiento informado (o contar con representante legal en caso necesario).

Privacidad y confidencialidad

Los datos personales recolectados durante la investigación serán utilizados exclusivamente con fines académicos. Se garantizará la privacidad de los participantes mediante consentimiento informado, y la confidencialidad de la información será protegida a través de la codificación de datos, almacenamiento seguro y acceso restringido únicamente al equipo investigador.

Apéndice C

Carta de consentimiento informado

Por medio de la firma del presente consentimiento informado **autoriza de manera completamente voluntaria** el uso de toda la información proporcionada durante esta entrevista y encuestas de seguimiento, así como de su información de contacto, en el desarrollo del trabajo de tesis titulado **Rediseño de los manipuladores de un robot de servicio de propósito general** elaborada por Rafael Cuéllar Ramírez y Marco Elian Soriano Pimentel, quienes fungen como investigadores principales y serán las únicas dos personas con acceso a la información recabada. El trabajo se desarrolla en el Laboratorio de Bio-Robótica adscrito al Departamento de Procesamiento de Señales de la División de Ingeniería Eléctrica de la Facultad de Ingeniería de la UNAM, como único centro de investigación participante.

El trabajo de tesis tiene como objetivo el diseño de los manipuladores y efectores finales de un robot de servicio de propósito general, así como el algoritmo de control, para permitirle tomar objetos de uso doméstico.

Como parte de la metodología de diseño centrada en el usuario de Karl T. Ulrich y Steven D. Eppinger, se requiere conocer la problemática que se pretende resolver a través de las experiencias de los usuarios potenciales y/o de aquellos que ya hayan utilizado tecnologías similares. Con este fin se realizará una entrevista inicial con una duración aproximada de una hora y encuestas de seguimiento posteriores. Durante la entrevista se grabará audio y se tomarán notas. Las encuestas de seguimiento se enviarán por medios electrónicos.

Su participación en este trabajo de investigación no representa riesgo de lesiones o problemas de salud relacionados, por lo tanto, no se contemplan normas de indemnización. Sin embargo, su participación conlleva los riesgos previsibles y no previsibles que se listan a continuación:

- Posible incomodidad al compartir información sobre necesidades personales de asistencia doméstica.
- Fatiga durante la sesión de entrevista.
- Riesgo mínimo de identificación indirecta a pesar de la codificación de datos.

Por otro lado, aunque **no se contempla una remuneración económica**, su participación conlleva los siguientes beneficios:

- Contribución directa al desarrollo de tecnología de asistencia robótica que puede mejorar la calidad de vida de personas con necesidades similares.
- Oportunidad de reflexionar sobre sus experiencias y necesidades, lo cual puede generar aprendizajes valiosos para su propio contexto.
- Retroalimentación que puede enriquecer su perspectiva sobre el diseño centrado en el usuario.
- Participación en investigación que busca cerrar la brecha entre el desarrollo tecnológico y las necesidades reales de los usuarios finales.

Se garantizará la privacidad de los participantes mediante consentimiento informado, y la confidencialidad de la información será protegida a través de la codificación de datos, almacenamiento seguro y acceso restringido únicamente al equipo investigador. La información será utilizada con fines estrictamente académicos y de investigación y, una vez concluido el trabajo, todas las grabaciones serán eliminadas.

Al autorizar su participación, el equipo investigador se compromete a compartir con usted los resultados a través de los trabajos derivados que sean publicados.

En caso de que tenga alguna duda o inquietud relacionada con su participación, puede contactar a los investigadores principales. Adicionalmente, **puede retirarse libremente en cualquier momento sin consecuencia alguna**, mientras la investigación esté en curso mediante la **Carta de revocación de consentimiento** adjunta.

El equipo de investigación se reserva el derecho de terminar su participación sin su consentimiento en caso de que la información proporcionada no resulte relevante para el desarrollo del tema de investigación.

Este proyecto está avalado por el **Comité de Ética en Investigación y Docencia de la Facultad de Ingeniería**.

Bibliografía

- [1] B. Siciliano and O. Khatib, “Robotics and the handbook,” in *Handbook of Robotics* (B. Siciliano and O. Khatib, eds.), ch. 1, pp. 1–6, Springer, 2da ed., 2016.
- [2] L. E. Sucar Succar, “Introducción,” in *Robótica de Servicio* (E. Sucar and Y. Hernández, eds.), ch. 1, pp. 1–5, Academia Mexicana de Computación, A.C., 2da ed., 2019.
- [3] J. Savage Carmona, “Robots de servicio,” in *Robótica de Servicio* (E. Sucar and Y. Hernández, eds.), ch. 2, pp. 7–23, Academia Mexicana de Computación, A.C., 2da ed., 2019.
- [4] T. Bajd and M. Mihelj, *Introduction to Robotics*. Springer, 1ra ed., 2013.
- [5] R. C. Arkin, *Behavior-Based Robotics*. MIT Press, 1ra ed., 1998.
- [6] B. Gates, “A robot in every home,” in *Scientific American, Inc.*, pp. 58–65, 2006.
- [7] J. Stücker and S. Behnke, “Integrating indoor mobility, object manipulation, and intuitive interaction for domestic service tasks,” in *2009 9th IEEE-RAS International Conference on Humanoid Robots*, pp. 506–513, 2009.
- [8] RoboCup Federation, “A brief history of RoboCup.” https://www.robocup.org/a_brief_history_of_robocup, 2025. Accessed: 2025-10-06.
- [9] RoboCup Federation, “RoboCup@Home League.” <https://athome.robocup.org/>, 2025. Official website of the RoboCup@Home league for autonomous service robots.
- [10] A. Bicchi and V. Kumar, “Robot grasping and contact: A review,” in *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings*, pp. 348–353, 2000.
- [11] M. Prats, P. J. Sanz, E. Martínez, R. Marín, and A. P. del Pobil, “Manipulación autónoma multipropósito en el robot de servicios jaume-2,” in *Revista Iberoamericana de Automática e Informática Industrial*, pp. 25–37, 2008.

- [12] BioRobotics Laboratory, UNAM, “Pumas OPL (Justina Robot).” [Online]. Available: <https://biorobotics.fi-p.unam.mx/robot-justina/>, 2025. Accessed: Oct. 1, 2025.
- [13] C. Peralta Vázquez, “Justina, robot de servicio creado en la unam.” <https://www.uv.mx/prensa/ciencia/justina-robot-de-servicio-creado-en-la-unam/>, 2019. Accessed: 2025-03-15.
- [14] UNAM Global, “En 30 años, robots con inteligencia artificial, que atenderían a los humanos.” https://unamglobal.unam.mx/global_revista/en-30-anos-creariamos-robots-con-inteligencia-artificial-que-atenderian-a-los-hum 2017. Accessed: 2026-08-19.
- [15] J. Savage, A. Llarena, G. Carrera, S. Cuellar, D. Esparza, Y. Minami, and U. Peñuelas, *ViRbot: A System for the Operation of Mobile Robots*, vol. 5001, pp. 512–519. 05 2007.
- [16] M. A. Negrete Villanueva, *Sistema de navegación para un robot de servicio*. Tesis de doctorado, Universidad Nacional Autónoma de México, Posgrado en Ciencia e Ingeniería de la Computación, Ciudad de México, México, 2019.
- [17] Open Robotics, “About ROS.”
- [18] Open Robotics, “ROS 2 Concepts: Nodes.” https://docs.ros.org/en/ros2_documentation/Concepts/Basic/About-Nodes.html. Accessed: 2026-08-11.
- [19] B. Gerkey, “Why ROS 2?.” https://design.ros2.org/articles/why_ros2.html, June 2014. Last modified: May 2022.
- [20] Open Robotics, “ROS 2 Concepts: Topics, Services, and Actions.”
- [21] J. J. Craig, *Introduction to Robotics: Mechanics and Control*. Upper Saddle River, NJ: Pearson Prentice Hall, 3 ed., 2005.
- [22] R. L. Norton, *Diseño de maquinaria: Síntesis y análisis de máquinas y mecanismos*. México, D.F.: McGraw-Hill Interamericana, 5 ed., 2013.
- [23] J. J. Uicker, G. R. Pennock, and J. E. Shigley, *Theory of Machines and Mechanisms*. New York: Oxford University Press, 3 ed., 2003.
- [24] F. Reuleaux, *The Kinematics of Machinery*. New York: Dover Publications, 1963.
- [25] T. Kivelä, *Increasing the Automation Level of Serial Robotic Manipulators with Optimal Design and Collision-free Path Control*. Doctoral dissertation, Tampere University of Technology, Tampere, Finland, 2017.
- [26] K. J. Waldron and J. Schmiedeler, “Kinematics,” in *Springer Handbook of Robotics* (B. Siciliano and O. Khatib, eds.), pp. 11–36, Springer, 2 ed., 2016.

- [27] Open Robotics, “Quaternion fundamentals.” <https://docs.ros.org/en/jazzy/Tutorials/Intermediate/Tf2/Quaternion-Fundamentals.html>, 2026. ROS 2 Jazzy documentation. Accessed: 2026-08-30.
- [28] J. Haviland and P. Corke, “Manipulator differential kinematics: Part ii: Acceleration and advanced applications,” *IEEE Robotics & Automation Magazine*, pp. 2–12, 2023.
- [29] G. Pahl, W. Beitz, J. Feldhusen, and K.-H. Grote, *Engineering Design: A Systematic Approach*. London: Springer-Verlag, 3rd ed., 2007. Traducción al inglés de la obra original en alemán *Konstruktionslehre*.
- [30] M. Garreta Domingo and E. Mor Pera, *Diseño centrado en el usuario*. Barcelona: Fundació per a la Universitat Oberta de Catalunya, 2013. Material didáctico UOC. PID_00176058.
- [31] D. A. Norman, *The Design of Everyday Things*. New York: Basic Books, revised and expanded ed., 2013. Primera edición publicada en 1988 como *The Psychology of Everyday Things*.
- [32] T. Lowdermilk, *User-Centered Design*. Sebastopol, CA: O’Reilly Media, 2013.
- [33] RoboCup@Home Technical Committee, “Robocup@home.” <https://athome.robocup.org/>, 2024. Accessed: 2025-03-15.
- [34] K. T. Ulrich and S. D. Eppinger, *Diseño y desarrollo de productos*. México, D.F.: McGraw-Hill/Interamericana Editores, S.A. de C.V., 5 ed., 2013. Traducción de la 5a. edición de *Product Design and Development*. Traducción: Jorge Humberto Romo Muñoz y Ricardo Martín Rubio Ruiz.